



Deusto

Facultad de Ingeniería
Ingeniaritza Fakultatea

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Desarrollo de una aplicación web para la generación automática de horarios universitarios basada en técnicas de automated timetabling

Gonzalo Hernández Idígoras

Director/a: Fernando Boto Sánchez

Donostia, junio de 2026

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Desarrollo de una aplicación web para la generación automática de horarios universitarios basada en técnicas de automated timetabling

Gonzalo Hernández Idígoras

Director/a: Fernando Boto Sánchez

Donostia, junio de 2026

Fernan
do Boto

Firmado digitalmente por
Fernando Boto
Nombre de reconocimiento
(DN): cn=Fernando Boto,
o=Universidad de Deusto,
ou=Area departamental de
Informática,
email=fernando.boto@deusto.e
s, c=ES
Fecha: 2026.06.17 08:29:00
+02'00'

Resumen

Este Trabajo de Fin de Grado se enmarca en el área de la investigación operativa y la inteligencia artificial, y se centra en el estudio y aplicación de técnicas de Automated Timetabling para la generación automática de horarios universitarios. El problema abordado, conocido como University Timetabling Problem, es un problema de optimización complejo que implica la satisfacción simultánea de múltiples restricciones, tales como la disponibilidad de profesores y aulas, la capacidad de los espacios, la asignación horaria y la evitación de solapamientos, buscando al mismo tiempo una solución de calidad.

El proyecto ha consistido en el análisis del problema y el estudio de diferentes enfoques de resolución existentes en la literatura y en sistemas similares. A partir de este análisis, se han definido los requisitos del sistema y se ha diseñado e implementado una aplicación web capaz de generar horarios de forma automática bajo un conjunto de restricciones predefinidas.

La herramienta desarrollada permite modelar el problema de planificación académica y obtener soluciones viables de forma automatizada, contribuyendo a reducir la carga de trabajo asociada a la elaboración manual de horarios.

El objetivo final del proyecto ha sido desarrollar una aplicación funcional que sirva como base para su posible uso e integración en entornos reales de planificación académica.

Descriptores

Automated Timetabling, University Timetabling, Optimización Combinatoria, Planificación Académica

Índice general

1	Introducción	1
1.1	Motivación y Justificación	1
1.2	Objetivos y Alcance	2
1.2.1	Objetivo Principal	2
1.2.2	Objetivos Específicos	2
1.2.3	Alcance del Proyecto	4
2	Estado del Arte	5
2.1	El Problema de Planificación Horaria Universitaria	5
2.2	Restricciones en la Generación de Horarios	5
2.3	Representación de Datos	6
2.3.1	Entidades Básicas del Problema	6
2.3.2	Estructuración de Recursos y Atributos	6
2.3.3	Gestión de datos y datasets en la planificación de horarios	7
2.4	Enfoques de Resolución Existentes	7
2.4.1	Métodos Basados en Investigación Operativa	8
2.4.2	Metaheurísticas	9
2.4.3	Técnicas Híbridas e Inteligentes	9
2.4.4	Hiper-heurísticas	10
2.5	Herramientas y Sistemas Existentes	10
2.5.1	International Timetabling Competition (ITC)	10
2.5.2	UniTime	11
2.5.3	FET	11
2.5.4	OptaPlanner	12
2.5.5	Otras Soluciones Comerciales	13

3	Planificación	14
3.1	Metodología	14
3.2	Fases del Proyecto	14
3.2.1	Planificación e Investigación	15
3.2.2	Definición y Diseño	15
3.2.3	Implementación y Desarrollo	16
3.2.4	Pruebas e Implantación	16
3.3	Presupuesto	18
3.3.1	Costes Laborales	18
3.3.2	Material y Equipo	19
3.3.3	Infraestructura	19
3.3.4	Coste Total del Proyecto	20
4	Desarrollo	21
4.1	Especificación de Requisitos	21
4.1.1	Requisitos Funcionales	21
4.1.2	Requisitos No Funcionales	22
4.2	Especificaciones de Diseño	23
4.2.1	Arquitectura General	23
4.2.2	Modelo de Datos	24
4.2.3	Diseño del Algoritmo	26
4.2.4	Diseño de la Interfaz	31
4.3	Consideraciones sobre la Implementación	31
4.3.1	Tecnologías Utilizadas	32
4.3.2	Estructura del Proyecto	33
4.3.3	Elección de Sistema de Almacenamiento	34
4.3.4	Estrategia de Generación	35
4.3.5	Implementación de Restricciones	37
4.3.6	Interfaz de Usuario	40
4.4	Plan de Pruebas	42
4.4.1	Casos de Prueba	43
5	Manual de Usuario	45
5.1	Gestión de Datos	45
5.1.1	Gestión de Aulas	45

5.1.2	Gestión de Docentes	46
5.1.3	Gestión de Grados y Asignaturas	47
5.2	Generación y Visualización de Horarios	48
6	Trabajo Futuro	51
6.1	Soporte para Grupos e Idiomas	51
6.2	Asignación de Aulas Específicas	52
6.3	Gestión de Asignaturas Optativas	53
6.4	Gestión de Horarios de Mañana y Tarde	54
6.5	Exportación e Integración de Horarios	55
6.6	Mejora del Proceso de Importación de Datos	56
7	Uso de la Inteligencia Artificial en el Proyecto	57
7.1	ChatGPT	57
7.2	NotebookLM	58
7.3	Gemini	58
7.4	GitHub Copilot	58
8	Valoración Ética del Proyecto	60
8.1	Beneficios Potenciales	60
8.2	Riesgos y Consideraciones Éticas	61
8.3	Reflexión Final	61
9	Conclusiones	62
10	Repositorio del Proyecto	64
	Bibliografía	65

Índice de figuras

2.1	Ejemplo de coloración de grafos. Fuente: Wikipedia	8
2.2	Interfaz de horario generado de UniTime. Fuente: UniTime	11
2.3	Interfaz de horario generado de FET. Fuente: Wikipedia	12
3.1	Diagrama de Gantt del proyecto	17
4.1	Diagrama de flujo de la fase de generación de horarios	29
4.2	Comparación de distintas soluciones generadas por el algoritmo (1º Ingeniería Informática)	29
4.3	Logs por consola obtenidos durante el proceso de optimización	31
4.4	Vista de árbol del proyecto	33
4.5	Formulario de creación de asignatura	41
4.6	Formulario de asignación de horas donde el docente no se encuentra disponible	42
5.1	Ventana de gestión de aulas	46
5.2	Ventana de gestión de docentes, filtrada en Ingeniería y mostrando el <i>dropdown</i> de asignaturas	47
5.3	Ventana de gestión de grados y asignaturas, filtrada en Ingeniería Informática	48
5.4	Ventana de generación de horarios	48
5.5	Modal de selección de grados	49
5.6	Visualización de horario de primer año de Ingeniería Informática	50
5.7	Auditoría de restricciones en una ejecución completa	50

Índice de cuadros

3.1	Tabla de presupuesto estimado del proyecto	20
4.1	Comparación entre distintas alternativas de almacenamiento	35
4.2	Penalizaciones asociadas a las restricciones blandas	39
4.3	Casos de prueba del sistema	44

Índice de Seudocódigos

4.1	Diccionario de restricciones que se importa en el algoritmo	37
-----	---	----

1. INTRODUCCIÓN

La planificación de horarios en cualquier ámbito constituye una tarea compleja que implica la coordinación de múltiples variables. En el contexto universitario, este proceso adquiere una especial relevancia debido a la necesidad de gestionar la disponibilidad del profesorado, la asignación de aulas y los grupos de estudiantes.

Tradicionalmente, la elaboración de horarios se ha realizado de forma manual o mediante herramientas con capacidades limitadas de automatización. Este enfoque puede resultar ineficiente, especialmente cuando aumenta el número de asignaturas, grupos o recursos disponibles que han de ser asignados. Además, la gestión manual incrementa la probabilidad de cometer errores y generar conflictos de asignación.

La generación de horarios académicos puede considerarse un problema de optimización sujeto a restricciones, en el que es necesario satisfacer simultáneamente múltiples condiciones y criterios de calidad. Entre las dificultades más habituales se encuentran la aparición de solapamientos entre clases, la distribución desigual de carga o la asignación de horarios poco adecuados para estudiantes y profesores.

Ante esta problemática resulta necesario disponer de herramientas capaces de automatizar este proceso de planificación, reducir la intervención manual y generar soluciones válidas en tiempos razonables. La aplicación de técnicas computacionales para la resolución de este tipo de problemas permite mejorar la eficiencia en la gestión de recursos y facilitar la obtención de horarios que satisfagan las restricciones establecidas por la institución.

1.1 MOTIVACIÓN Y JUSTIFICACIÓN

La elaboración de horarios académicos no solo tiene un impacto en la gestión eficiente de los recursos universitarios, sino también en la experiencia diaria de estudiantes y docentes. Un horario poco equilibrado puede generar jornadas fragmentadas, con largos periodos de espera entre clases o una distribución desigual de la carga lectiva entre distintos días. Estas situaciones pueden afectar negativamente a la satisfacción de los estudiantes y, en algunos casos, repercutir negativamente en su motivación y asistencia a las actividades académicas.

A pesar de la importancia de este proceso, la planificación de horarios continúa realizándose de forma manual o con un elevado grado de intervención humana. La necesidad de coordinar numerosas restricciones y preferencias convierte esta tarea en un proceso complejo, laborioso y susceptible de generar configuraciones que, aun siendo válidas desde un punto de vista organizativo, no siempre resultan satisfactorias para los usuarios

finales.

La motivación de este proyecto surge de una experiencia personal durante mis años como estudiante en la Universidad de Deusto. A lo largo de este periodo, he tenido la oportunidad de convivir con horarios académicos de distinta naturaleza, algunos de los cuales presentaban clases partidas, largos intervalos entre sesiones o desplazamientos entre diferentes puntos del campus. Estas situaciones me permitieron comprender tanto el impacto que un horario puede tener en la experiencia del estudiante como la dificultad inherente al proceso de planificación.

Partiendo de esta realidad, el presente proyecto explora la aplicación de técnicas computacionales para automatizar la generación de horarios académicos, con el objetivo de facilitar las tareas de planificación y obtener distribuciones que satisfagan las restricciones establecidas al mismo tiempo que mejoran la calidad de los horarios generados.

1.2 OBJETIVOS Y ALCANCE

1.2.1 Objetivo Principal

El objetivo principal de este proyecto es diseñar y desarrollar una solución informática que facilite y mejore los procesos de planificación académica de la Universidad de Deusto. Para ello, se propone una alternativa que permita automatizar la generación de horarios académicos, optimizando la gestión de recursos disponibles y sirviendo como base para una posible implantación futura dentro del entorno universitario.

Además, se persigue contribuir a la mejora de la experiencia académica de los estudiantes y del personal responsable de la planificación, proporcionando una herramienta capaz de generar horarios válidos y adaptados a las necesidades y restricciones establecidas por la universidad.

1.2.2 Objetivos Específicos

Con el propósito de alcanzar el objetivo principal del proyecto, se han definido una serie de objetivos específicos que abarcan tanto los aspectos relacionados con la generación automática de horarios como aquellos orientados a la experiencia de uso de la aplicación.

1.2.2.1 Objetivos funcionales

Los principales objetivos funcionales del sistema son los siguientes:

- Diseñar un modelo de datos capaz de representar adecuadamente los distintos elementos involucrados en la planificación de horarios.
- Identificar y modelar las restricciones que intervienen en la generación de horarios.
- Diferenciar entre restricciones duras, cuyo incumplimiento invalida una solución, y restricciones blandas, empleadas para evaluar la calidad de las soluciones propuestas.
- Facilitar la adaptación del sistema ante modificaciones en los datos de entrada o las restricciones consideradas.
- Desarrollar un mecanismo capaz de generar automáticamente horarios académicos válidos a partir de los datos y restricciones definidos.

1.2.2.2 Objetivos de usabilidad

Los objetivos definidos no se limitan exclusivamente a la generación automática de horarios académicos. Además de garantizar la obtención de soluciones válidas, el proyecto busca proporcionar una experiencia de uso satisfactoria para los responsables de la planificación. En consecuencia, se ha considerado fundamental desarrollar una aplicación intuitiva y fácil de utilizar, que simplifique la gestión de los datos y facilite la consulta e interpretación de la información generada.

Con el propósito de alcanzar estos objetivos, durante el desarrollo de la aplicación se han tenido en consideración los siguientes aspectos:

- Diseñar una interfaz web intuitiva y accesible, orientada a usuarios sin conocimientos técnicos avanzados.
- Facilitar la visualización y consulta de la información almacenada en el sistema.
- Ofrecer una representación clara y comprensible de los horarios generados por la aplicación.
- Simplificar y agilizar las tareas de creación, modificación y gestión de los datos necesarios para el correcto funcionamiento de la aplicación.

1.2.3 Alcance del Proyecto

El presente proyecto tiene como finalidad el diseño y desarrollo de una aplicación web orientada a la planificación académica universitaria. La solución desarrollada permite gestionar la información necesaria para la generación de horarios, así como automatizar el proceso de creación de estos a partir de un conjunto de restricciones previamente definidas.

Dentro del alcance del proyecto se incluye el desarrollo de una interfaz web que facilite la administración de los distintos elementos involucrados en la planificación académica, tales como asignaturas, grupos, aulas, docentes y demás información requerida por el sistema. Asimismo, la aplicación proporciona mecanismos para la visualización y consulta de los datos almacenados, permitiendo a los usuarios gestionar la información de forma sencilla e intuitiva.

Además, se contempla el desarrollo e integración de un motor de generación de horarios capaz de producir soluciones válidas, respetando las restricciones establecidas. Los resultados obtenidos pueden ser consultados a través de la propia aplicación mediante representaciones visuales diseñadas para facilitar su interpretación y análisis.

No obstante, el alcance del proyecto se limita al desarrollo de un prototipo funcional orientado a validar la viabilidad de la propuesta. Aspectos como la implantación en entornos de producción, la integración con sistemas corporativos existentes o la incorporación de funcionalidades avanzadas quedan fuera del alcance del presente trabajo y se plantean como posibles líneas de evolución futura.

2. ESTADO DEL ARTE

2.1 EL PROBLEMA DE PLANIFICACIÓN HORARIA UNIVERSITARIA

La generación automática de horarios [1] académicos constituye uno de los problemas clásicos de optimización combinatoria dentro del ámbito de la investigación operativa [2] y la inteligencia artificial. Este problema, conocido habitualmente como *University Course Timetabling Problem* (UCTP) [3], consiste en asignar un conjunto de actividades académicas a intervalos temporales y espacios físicos disponibles, garantizando el cumplimiento de un conjunto de restricciones previamente definidas.

El objetivo principal es construir horarios válidos que permitan organizar las clases de una institución educativa evitando conflictos entre asignaturas, profesorado, estudiantes y aulas. Debido al elevado número de combinaciones posibles y a la gran cantidad de restricciones involucradas, el problema ha sido clasificado como NP-Hard, lo que dificulta la obtención de soluciones óptimas mediante métodos exactos para instancias de tamaño real.

2.2 RESTRICCIONES EN LA GENERACIÓN DE HORARIOS

Las restricciones utilizadas en los problemas de planificación horaria suelen clasificarse en dos categorías principales:

- **Restricciones duras (Hard Constraints):** Son aquellas que deben cumplirse obligatoriamente para que una solución sea considerada válida. Entre ellas se encuentran la imposibilidad de asignar dos clases al mismo aula en el mismo horario, evitar que un profesor imparta dos asignaturas simultáneamente o garantizar que cada actividad disponga de un aula adecuada.
- **Restricciones blandas (Soft Constraints):** Representan preferencias o criterios de calidad cuya violación es aceptable, aunque debe minimizarse. Algunos ejemplos son la distribución equilibrada de clases durante la semana, la reducción de huecos en los horarios o las preferencias horarias del profesorado.

La calidad final de un horario suele evaluarse mediante funciones de penalización asociadas al incumplimiento de restricciones blandas, mientras que las restricciones duras deben satisfacerse completamente.

2.3 REPRESENTACIÓN DE DATOS

La correcta modelización del problema de planificación horaria no depende únicamente de las técnicas de optimización empleadas, sino también de la forma en la que se estructuran y representan los datos de entrada. Un modelo de datos bien definido permite expresar de manera precisa las restricciones del sistema, facilita la implementación de los algoritmos de resolución y mejora la escalabilidad del enfoque adoptado. Por este motivo, la representación de la información constituye un aspecto fundamental dentro del estado del arte en este tipo de problemas.

2.3.1 Entidades Básicas del Problema

En el contexto del problema de planificación horaria universitaria, los datos suelen organizarse en torno a un conjunto de entidades fundamentales que permiten describir todos los elementos involucrados en la generación del horario.

- **Evento:** Representa una actividad académica que debe ser programada, como una clase teórica, una sesión práctica o un examen.
- **Timeslot (intervalo de tiempo):** Define una franja temporal discreta en la que puede asignarse un evento, como una hora concreta dentro del calendario semanal.
- **Recurso:** Engloba todos los elementos necesarios para la ejecución del evento, incluyendo profesores, estudiantes, aulas o equipamiento específico.
- **Restricción (Constraint):** Conjunto de condiciones que deben cumplirse para garantizar la validez del horario, como la disponibilidad del profesorado, la capacidad de las aulas o la incompatibilidad entre eventos.

Estas entidades constituyen la base sobre la que se construyen tanto los modelos teóricos como las implementaciones prácticas de los sistemas de planificación automática.

2.3.2 Estructuración de Recursos y Atributos

Una representación más detallada de los recursos puede realizarse mediante la definición de atributos que describen sus características funcionales y técnicas. Este enfoque permite una modelización más flexible y expresiva, facilitando la incorporación de restricciones complejas.

- **Atributos simples:** Valores *booleanos* que indican la presencia o ausencia de una característica, como la accesibilidad para personas con movilidad reducida o la disponibilidad de proyector en un aula.
- **Atributos de lista:** Permiten asignar múltiples valores dentro de un conjunto predefinido, como los posibles usos de un aula (laboratorio, aula teórica, sala de informática, etc.).
- **Atributos exclusivos:** Permiten seleccionar un único valor dentro de un conjunto de opciones, como el edificio en el que se encuentra un aula.
- **Atributos numéricos:** Representan valores enteros positivos que describen propiedades cuantitativas, como la capacidad máxima de estudiantes de un aula.

Este tipo de estructuración permite establecer relaciones más ricas entre eventos y recursos, mejorando la capacidad del sistema para representar restricciones reales de un entorno académico.

2.3.3 Gestión de datos y datasets en la planificación de horarios

La gestión de datos constituye otro elemento clave en el problema de planificación horaria, ya que la calidad del modelo depende directamente de la información utilizada como entrada. En la práctica, los datasets pueden incluir información heterogénea como asignaturas, grupos de estudiantes, docentes, aulas, franjas horarias y restricciones asociadas.

En la literatura se distinguen habitualmente dos tipos de datasets. Por un lado, los conjuntos de datos sintéticos, diseñados específicamente para evaluar el comportamiento de los algoritmos en escenarios controlados. Por otro lado, los datos reales procedentes de instituciones educativas, que ofrecen una mayor fidelidad al problema, aunque también introducen ruido, inconsistencias y restricciones implícitas más difíciles de modelar.

Una correcta estructuración del dataset es esencial para garantizar la aplicabilidad de los algoritmos de resolución. En este sentido, los modelos de datos bien diseñados permiten desacoplar la representación del problema de los métodos de optimización, facilitando la reutilización del sistema en distintos contextos educativos y mejorando su escalabilidad.

2.4 ENFOQUES DE RESOLUCIÓN EXISTENTES

A lo largo de las últimas décadas se han propuesto numerosas técnicas para resolver el problema de planificación horaria universitaria.

2.4.1 Métodos Basados en Investigación Operativa

Los primeros enfoques se fundamentaron en técnicas clásicas de investigación operativa, entre las que destacan la coloración de grafos [4], la programación lineal [5] y entera [6], y los problemas de satisfacción de restricciones (CSP) [7].

2.4.1.1 Coloración de Grafos

Representa los eventos como nodos y los conflictos o restricciones como aristas. El objetivo es asignar colores (franja horaria) de modo que ningún nodo adyacente comparta el mismo color. Aunque este enfoque permite representar el problema de forma intuitiva, la búsqueda de coloraciones óptimas presenta una elevada complejidad computacional.

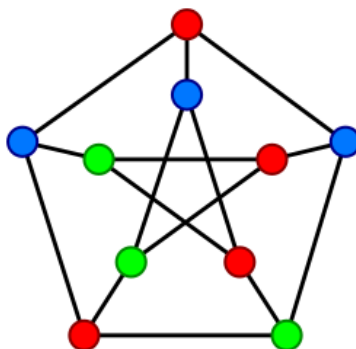


Figura 2.1: Ejemplo de coloración de grafos. Fuente: Wikipedia

2.4.1.2 Programación Entera y Lineal

Utiliza formulaciones matemáticas para maximizar beneficios o minimizar costos (descontento de estudiantes/profesores). Estos modelos permiten obtener soluciones exactas en instancias reducidas, aunque su escalabilidad resulta limitada cuando aumenta el tamaño del problema.

2.4.1.3 Técnicas Basadas en Restricciones (CSPs)

Los modelos CSP representan cada decisión de planificación como una variable con un dominio de valores posibles y un conjunto de restricciones que deben satisfacerse. Este enfoque resulta especialmente adecuado para problemas donde predominan restricciones complejas y altamente interdependientes.

2.4.2 Metaheurísticas

Debido a la complejidad del problema, gran parte de la investigación reciente se ha centrado en el uso de metaheurísticas [8] capaces de obtener soluciones de alta calidad en tiempos razonables.

Entre las técnicas más utilizadas destacan los algoritmos genéticos, los sistemas de colonia de hormigas, la búsqueda tabú y el recocido simulado (*Simulated Annealing*). Estas metodologías exploran el espacio de soluciones mediante estrategias de búsqueda guiadas que permiten escapar de óptimos locales y mejorar progresivamente la calidad de los horarios generados.

Aunque estas técnicas no garantizan la obtención de soluciones óptimas, han demostrado ofrecer resultados competitivos en problemas de gran tamaño y con múltiples restricciones.

2.4.3 Técnicas Híbridas e Inteligentes

Las investigaciones más recientes combinan diferentes estrategias con el objetivo de aprovechar las ventajas de cada una de ellas. Estos enfoques híbridos integran métodos de satisfacción de restricciones con metaheurísticas, algoritmos evolutivos con búsquedas locales o técnicas basadas en conocimiento experto.

La combinación de distintos métodos ha permitido mejorar significativamente la calidad de las soluciones obtenidas y constituye una de las líneas de investigación más activas dentro del ámbito de la planificación automática de horarios.

Estos enfoques buscan superar las debilidades de los métodos individuales combinándolos o utilizando inteligencia artificial avanzada:

- **Enfoques Combinatorios (Híbridos):** Combinan las fortalezas de diferentes algoritmos. Por ejemplo, usar programación por restricciones para generar una solución inicial y luego recocido simulado para mejorarla.
- **Lógica Difusa (Fuzzy Logic):** Se utiliza para manejar la incertidumbre y la ambigüedad en las restricciones, permitiendo una evaluación más flexible de la calidad de los horarios.
- **Razonamiento Basado en Casos (CBR):** Selecciona las mejores heurísticas basándose en situaciones de resolución de problemas similares encontradas anteriormente.

2.4.4 Hiper-heurísticas

[9] Un enfoque más reciente que se define como "heurísticas para elegir heurísticas". A diferencia de las metaheurísticas convencionales, las hiper-heurísticas no operan directamente sobre el espacio de soluciones del problema, sino sobre un espacio de búsqueda de heurísticas. Se dividen en dos categorías principales:

- **Selección de Heurísticas:** Metodologías para elegir heurísticas existentes de bajo nivel (ya sean constructivas o perturbativas) de forma inteligente durante el proceso de búsqueda.
- **Generación de Heurísticas:** Metodologías para crear nuevas heurísticas a partir de componentes básicos, frecuentemente utilizando programación genética.

2.5 HERRAMIENTAS Y SISTEMAS EXISTENTES

A día de hoy existen múltiples herramientas y plataformas orientadas a la resolución del problema de planificación automática de horarios académicos. Estas soluciones han servido tanto como entornos de investigación para la evaluación de algoritmos como herramientas prácticas utilizadas por instituciones educativas para gestionar sus horarios. El análisis de estos sistemas permite comprender las diferentes estrategias empleadas para modelar restricciones, optimizar soluciones y gestionar la complejidad inherente al problema. A continuación, se presentan algunas de las herramientas y proyectos más representativos en este ámbito.

2.5.1 International Timetabling Competition (ITC)

La *International Timetabling Competition* [10] es una competición internacional creada con el objetivo de proporcionar conjuntos de datos estandarizados y criterios de evaluación comunes para comparar algoritmos de generación automática de horarios. Desde su primera edición, la ITC se ha convertido en una referencia dentro de la comunidad científica dedicada al estudio de los problemas de planificación académica.

Los conjuntos de datos propuestos por la competición incluyen diferentes tipos de restricciones duras y blandas que permiten evaluar la calidad de las soluciones obtenidas por distintos algoritmos. Gracias a esta iniciativa, los investigadores pueden comparar de forma objetiva nuevas técnicas de resolución y medir su rendimiento sobre problemas de características similares.

La ITC ha contribuido significativamente al avance del estado del arte en planificación automática, ya que muchos de los algoritmos actuales han sido desarrollados y validados utilizando las instancias proporcionadas

por esta competición.

2.5.2 UniTime

UniTime [11] es una plataforma de código abierto desarrollada originalmente por la Universidad de Purdue para la gestión integral de horarios académicos universitarios. El sistema permite planificar asignaturas, gestionar aulas, coordinar recursos y generar horarios considerando un amplio conjunto de restricciones institucionales.

Una de las principales características de UniTime es su capacidad para modelar problemas complejos mediante técnicas de optimización basadas en programación por restricciones y búsqueda local. Además, ofrece soporte para la gestión completa del ciclo académico, incluyendo planificación de exámenes y asignación de espacios.

Debido a su madurez y amplia adopción en instituciones educativas, UniTime constituye una de las referencias más importantes dentro del ámbito de la planificación académica automatizada.

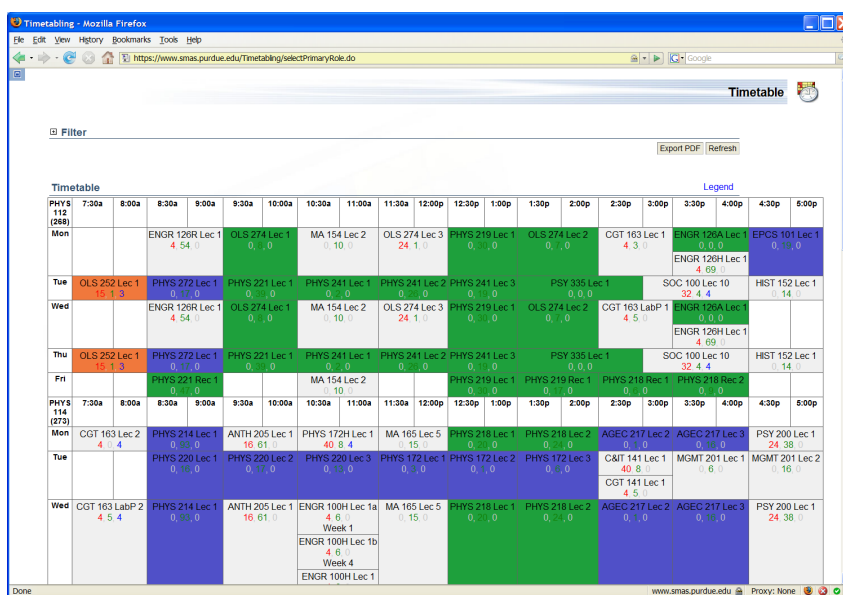


Figura 2.2: Interfaz de horario generado de UniTime. Fuente: UniTime

2.5.3 FET

FET [12, 13] es una herramienta de software libre diseñada específicamente para la generación automática de horarios en centros educativos. Su principal objetivo es proporcionar una solución sencilla y accesible para escuelas, institutos y universidades que necesiten automatizar la planificación de clases.

El sistema permite definir profesores, asignaturas, grupos de estudiantes, aulas y múltiples restricciones relacionadas con la disponibilidad y organización de las actividades académicas. A partir de esta información, FET utiliza algoritmos de búsqueda para construir horarios válidos que satisfacen las restricciones establecidas.

Su facilidad de uso, junto con su carácter gratuito y multiplataforma, ha favorecido su utilización en numerosos centros educativos de todo el mundo.

	Lunes	Martes	Miércoles	Jueves	Viernes	Sábado
08:00	M2 C G101 A1	-x-	M2 C G103 A3	-x-	M2 C G102 A2	
09:45	M2 C G103 A3	-x-	M2 C G102 A2	-x-		
11:30	M2 C G102 A2	-x-	M2 C G101 A1	-x-	-x-	
13:30	-x-	-x-	-x-	-x-	M2 C G101 A1	
15:15	-x-	-x-	-x-	-x-	M2 C G103 A3	
17:00	-x-	-x-	-x-	-x-	-x-	-x-

Figura 2.3: Interfaz de horario generado de FET. Fuente: Wikipedia

2.5.4 OptaPlanner

OptaPlanner [14] es una biblioteca de optimización desarrollada en Java orientada a la resolución de problemas complejos de planificación y asignación de recursos. Aunque no está diseñada exclusivamente para la generación de horarios académicos, proporciona herramientas específicas para abordar este tipo de problemas mediante técnicas de optimización combinatoria.

La plataforma permite modelar restricciones personalizadas y aplicar diferentes algoritmos metaheurísticos, como búsqueda local, *Simulated Annealing*, *Tabu Search* o algoritmos genéticos. Gracias a esta flexibilidad, OptaPlanner puede adaptarse a escenarios muy diversos y de gran escala.

Su integración con aplicaciones empresariales y su capacidad para gestionar problemas de elevada complejidad la convierten en una solución ampliamente utilizada tanto en entornos académicos como industriales.

2.5.5 Otras Soluciones Comerciales

Además de las herramientas académicas y de código abierto, existen diversas soluciones comerciales especializadas en la planificación de horarios educativos. Entre las más conocidas se encuentran Untis, aSc Timetables, Horarium y Omniscol [15, 16, 17, 18].

Estas plataformas suelen ofrecer interfaces gráficas avanzadas, funcionalidades de gestión administrativa y mecanismos de generación automática de horarios adaptados a las necesidades específicas de cada institución. Asimismo, incorporan herramientas para la publicación de horarios, comunicación con estudiantes y gestión de cambios durante el curso académico.

Aunque estas soluciones presentan un elevado grado de funcionalidad y facilidad de uso, generalmente requieren licencias comerciales y ofrecen menores posibilidades de personalización que las herramientas de código abierto o los desarrollos específicos realizados a medida.

3. PLANIFICACIÓN

3.1 METODOLOGÍA

Para el desarrollo de este proyecto se ha seguido una metodología de trabajo de carácter iterativo e incremental, inspirada en los principios de las metodologías ágiles, concretamente en Scrum [19]. No obstante, debido al contexto académico del proyecto, esta metodología ha sido adaptada, prescindiendo de ciertos elementos formales como la definición de roles específicos o el uso de herramientas de gestión de *sprints*.

El trabajo se organizó en ciclos de desarrollo de aproximadamente dos semanas de duración, equivalentes a *sprints*. Al inicio de cada iteración se establecían los objetivos y tareas a desarrollar durante ese periodo, priorizando las funcionalidades a implementar o mejorar en función del estado del proyecto y las necesidades detectadas.

Al final de cada ciclo se realizaban reuniones periódicas con el tutor del Trabajo Fin de Grado, con una frecuencia aproximada de 14 días. En estas reuniones se revisaba el progreso alcanzado, se analizaban los resultados obtenidos y se definían los siguientes pasos a seguir. Este proceso de revisión permitía validar la dirección del proyecto de forma continua y realizar ajustes sobre la planificación inicial cuando era necesario.

Este enfoque iterativo ha resultado especialmente adecuado para el desarrollo del proyecto, ya que la naturaleza del problema de generación de horarios requiere un proceso de refinamiento progresivo. Tanto el diseño del algoritmo como la definición de las restricciones y la estructura de datos han evolucionado a lo largo del desarrollo, incorporando mejoras derivadas de las revisiones periódicas.

En consecuencia, la metodología seguida ha permitido mantener un desarrollo flexible, facilitando la incorporación de cambios y la mejora progresiva de la solución implementada, sin perder la trazabilidad del avance del proyecto.

3.2 FASES DEL PROYECTO

El desarrollo del proyecto se ha estructurado en cuatro fases principales, que permiten organizar el trabajo de forma secuencial y coherente. Cada fase persigue un objetivo específico dentro del ciclo de vida del sistema y agrupa un conjunto de tareas orientadas a su consecución.

3.2.1 Planificación e Investigación

La primera fase del proyecto estuvo dedicada a la planificación del trabajo y al estudio del problema de generación automática de horarios académicos. Antes de iniciar el diseño y desarrollo de la aplicación, resultó necesario comprender las características del problema, identificar los principales desafíos asociados a la planificación académica y analizar las soluciones existentes propuestas tanto en el ámbito académico como en el profesional.

Con este objetivo, se llevó a cabo una revisión bibliográfica de artículos científicos, documentación técnica y trabajos relacionados con el problema de planificación automática de horarios (*Automated Timetabling*) [1, 3]. Asimismo, se analizaron diferentes soluciones y herramientas existentes con el fin de conocer los enfoques más utilizados para la representación de restricciones y la generación de horarios académicos.

El conocimiento adquirido durante esta fase permitió establecer los requisitos iniciales del sistema, definir el modelo de datos y seleccionar la estrategia de resolución que posteriormente sería implementada en la aplicación. Además, sirvió como base para la elaboración del estado del arte presentado en el Capítulo 2.

3.2.2 Definición y Diseño

Una vez comprendido el problema y analizadas las soluciones existentes, la siguiente fase del proyecto estuvo orientada a la definición de los requisitos y al diseño de la solución propuesta. El objetivo principal consistió en establecer una base sólida sobre la que desarrollar posteriormente la aplicación, identificando tanto las necesidades funcionales como las decisiones técnicas necesarias para su implementación.

Durante esta etapa se definieron los requisitos funcionales y no funcionales del sistema, así como las principales restricciones que debían considerarse durante la generación de horarios. Asimismo, se diseñó el modelo de datos encargado de representar las entidades involucradas en el problema, incluyendo asignaturas, docentes, aulas y demás elementos necesarios para la planificación académica.

De forma paralela, se estudió la estrategia de generación más adecuada para las características específicas del problema abordado. Este proceso permitió definir la estructura general del algoritmo, establecer el tratamiento de las restricciones y determinar los mecanismos necesarios para evaluar la calidad de las soluciones generadas.

Las decisiones adoptadas durante esta fase sirvieron como guía para el desarrollo posterior de la aplicación y permitieron disponer de una arquitectura coherente y alineada con los objetivos del proyecto.

3.2.3 Implementación y Desarrollo

La fase de implementación estuvo dedicada a la construcción de la aplicación a partir de los diseños y especificaciones definidos previamente. El desarrollo se realizó de forma iterativa e incremental, siguiendo la metodología de trabajo adoptada para el proyecto y permitiendo incorporar mejoras progresivamente a medida que avanzaba el desarrollo.

Durante esta etapa se implementaron tanto los componentes encargados de la gestión de datos como el algoritmo responsable de la generación automática de horarios. Asimismo, se desarrolló la interfaz web que permite a los usuarios administrar la información necesaria para el funcionamiento del sistema y visualizar los resultados obtenidos.

Paralelamente, se llevó a cabo la implementación de las restricciones consideradas por el algoritmo y la preparación de los datos utilizados durante las pruebas. Este trabajo incluyó la recopilación, organización y adaptación de la información académica necesaria para construir un conjunto de datos representativo del entorno universitario objeto de estudio.

Como resultado de esta fase se obtuvo una aplicación funcional capaz de gestionar la información académica y generar automáticamente horarios válidos de acuerdo con las restricciones definidas.

3.2.4 Pruebas e Implantación

La última fase del proyecto estuvo orientada a la validación del sistema desarrollado y a la comprobación de que las funcionalidades implementadas cumplieran los objetivos establecidos inicialmente.

Con este propósito, se realizaron diferentes pruebas destinadas a verificar el correcto funcionamiento de los distintos componentes de la aplicación. En particular, se evaluó el comportamiento del algoritmo de generación, comprobando que las soluciones obtenidas respetaban las restricciones consideradas obligatorias y que el sistema era capaz de producir resultados consistentes a partir de los datos proporcionados.

Además, se revisó el funcionamiento de los mecanismos de gestión de datos y de la interfaz de usuario, identificando posibles errores y aplicando las correcciones necesarias para mejorar la estabilidad y la experiencia de uso de la aplicación.

Los resultados obtenidos durante esta fase permitieron confirmar la viabilidad de la solución desarrollada y verificar que el sistema era capaz de cumplir los objetivos definidos para el proyecto.



Figura 3.1: Diagrama de Gantt del proyecto

3.3 PRESUPUESTO

El coste es un elemento clave en todo proyecto, ya que permite realizar una estimación económica de los recursos necesarios para su desarrollo. En este caso, al tratarse de un proyecto académico, el presupuesto se ha elaborado a partir de una estimación del esfuerzo invertido en horas de trabajo y del uso de recursos básicos necesarios para su desarrollo.

3.3.1 Costes Laborales

El estudiante ha asumido a lo largo del proyecto distintos roles, incluyendo tareas de análisis, diseño, implementación, pruebas y documentación. Para la estimación del coste asociado a su trabajo, se ha considerado una dedicación aproximada de 250 horas de desarrollo efectivo.

Se ha utilizado una tarifa estimada de 20 €/hora, representativa de un perfil junior en formación en el ámbito del desarrollo de software.

Por tanto, el coste asociado al trabajo realizado se calcula como:

- Horas trabajadas: 250 horas
- Coste por hora: 20 €/hora

Coste del estudiante = $250 \times 20 = 5.000$ €

Adicionalmente, es importante señalar que durante el desarrollo del proyecto se identificaron funcionalidades y mejoras que, aunque no se incluyeron dentro del alcance final, habrían requerido un esfuerzo adicional estimado de aproximadamente 100 horas más de trabajo. Estas tareas adicionales incluyen la definición, discusión e integración de soluciones más complejas o extensiones funcionales del sistema.

En caso de haberse implementado, este esfuerzo adicional supondría:

- Horas adicionales estimadas: 100 horas
- Coste por hora: 20 €/hora

Coste adicional estimado = $100 \times 20 = 2.000$ €

De este modo, el esfuerzo total potencial del proyecto ascendería a 350 horas, con un coste equivalente de **7.000**

€, aunque el alcance final del proyecto se ha limitado a la estimación inicial de 250 horas.

3.3.2 Material y Equipo

Para el desarrollo del proyecto se ha utilizado exclusivamente equipamiento informático estándar, junto con diversas herramientas de software necesarias para la programación, diseño y documentación del sistema.

En cuanto al software empleado, este corresponde a herramientas de desarrollo de uso gratuito, por lo que no ha supuesto un coste económico directo dentro del proyecto.

Dado que el ordenador utilizado constituye el principal recurso físico del proyecto, se considera adecuada la estimación de su coste mediante un valor medio de mercado para un equipo de características estándar en entornos de desarrollo. Para ello, se toma como referencia un ordenador de empresa con un coste aproximado de **1.000 €**.

Por tanto, el coste asociado a material y equipo se estima en:

- Ordenador: 1.000 €
- Software y Herramientas: 0 €

Coste total de material y equipo: 1.000 €

3.3.3 Infraestructura

En caso de que la aplicación desarrollada se desplegara en un entorno real de producción, sería necesario contemplar los costes asociados a la infraestructura de alojamiento y mantenimiento del sistema.

Estos costes incluirían, principalmente, el servicio de hosting del servidor donde se aloja la aplicación, así como el dominio asociado al servicio.

A modo de estimación, se consideran valores habituales de mercado:

- Hosting de Servidor: 15 €/mes
- Dominio web: 10 €/año

Coste anual total de infraestructura: 190 €

3.3.4 Coste Total del Proyecto

A continuación se presenta un resumen global de los costes estimados del proyecto, agrupando los elementos mencionados en los apartados anteriores.

Nombre del Coste	Dinero
Costes Laborales	7.000 €
Material y Equipo	1.000 €
Infraestructura	190 €
TOTAL	8.190 €

Cuadro 3.1: Tabla de presupuesto estimado del proyecto

Este presupuesto refleja una estimación global del coste que supondría el desarrollo y despliegue del sistema en un entorno real, considerando tanto el esfuerzo humano como los recursos materiales e infraestructura necesarios.

4. DESARROLLO

En este capítulo se describe el proceso de desarrollo de la solución propuesta en el presente proyecto. Para ello, se presentan los requisitos que debe cumplir para satisfacer los objetivos previamente mencionados, las tecnologías utilizadas durante su desarrollo y las funcionalidades implementadas en la aplicación.

4.1 ESPECIFICACIÓN DE REQUISITOS

Antes de abordar el diseño e implementación de la solución propuesta, resulta necesario definir los requisitos que debe satisfacer el sistema. La especificación de requisitos permite identificar las necesidades y restricciones que han guiado el desarrollo de la aplicación, estableciendo las funcionalidades esperadas y las características que debe cumplir para alcanzar los objetivos planteados.

4.1.1 Requisitos Funcionales

Los requisitos funcionales describen las funcionalidades que debe proporcionar el sistema para satisfacer las necesidades identificadas durante el análisis del problema. Estos requisitos definen las operaciones que pueden realizar los usuarios, así como los procesos que debe ejecutar la aplicación para garantizar el correcto funcionamiento de la misma.

- **RF1.** El sistema deberá permitir la gestión de asignaturas, incluyendo su creación, modificación y eliminación.
- **RF2.** El sistema deberá permitir la gestión de docente y la asignación de los mismos a las asignaturas correspondientes.
- **RF3.** El sistema deberá permitir la gestión de aulas y sus capacidades asociadas.
- **RF4.** El sistema deberá permitir la importación de datos, para agilizar el proceso de creación de datos.
- **RF5.** El sistema deberá permitir la gestión de grupos académicos y su relación con las asignaturas impartidas.
- **RF6.** El sistema deberá generar horarios académicos a partir de la información y restricciones definidas.

- **RF7.** El sistema deberá descartar aquellas soluciones que no cumplan las restricciones consideradas obligatorias, es decir, las restricciones duras.
- **RF8.** El sistema deberá evaluar las soluciones ofrecidas mediante una valoración basada en las restricciones de calidad, es decir, las restricciones blandas.
- **RF9.** El sistema deberá permitir la visualización de los horarios generados mediante una representación visual comprensible.
- **RF10.** El sistema deberá permitir la modificación de los datos utilizados para la generación de horarios.
- **RF11.** El usuario deberá poder seleccionar el semestre académico para el cual desea generar el horario.
- **RF12.** El usuario deberá poder seleccionar las restricciones blandas que el algoritmo tendrá en cuenta durante el proceso de optimización de horarios.

4.1.2 Requisitos No Funcionales

Los requisitos no funcionales establecen las características de calidad y las restricciones técnicas que debe cumplir el sistema. Estos describen aspectos relacionados con la usabilidad, rendimiento, seguridad o escalabilidad de la aplicación.

- **RNF1.** La aplicación deberá proporcionar una interfaz web intuitiva y fácil de utilizar por los usuarios sin conocimientos técnicos avanzados.
- **RNF2.** La arquitectura de la aplicación deberá facilitar la incorporación de nuevas funcionalidades en el futuro.
- **RNF3.** La arquitectura de la aplicación deberá facilitar la adición de restricciones blandas en el futuro.
- **RNF4.** La aplicación deberá presentar la información de forma clara y comprensible para los usuarios.
- **RNF5.** El sistema deberá minimizar la intervención manual necesaria durante el proceso de generación de horarios.
- **RNF6.** La aplicación deberá ofrecer interfaces que permitan gestionar los datos de forma rápida.
- **RNF7.** El algoritmo deberá proporcionar soluciones en un tiempo razonable.

- **RNF8.** El algoritmo deberá minimizar el número de ejecuciones donde no encuentra una solución.

4.2 ESPECIFICACIONES DE DISEÑO

Una vez establecidos los requisitos que debe satisfacer el sistema, resulta necesario definir las decisiones de diseño adoptadas para su implementación. En este apartado se describen los principales aspectos relacionados con la arquitectura de la aplicación, la organización de los datos, el diseño del algoritmo de generación de horarios y la interfaz de usuario desarrollada. Estas decisiones constituyen la base sobre la que se ha construido la solución propuesta y permiten comprender su funcionamiento interno.

4.2.1 Arquitectura General

La solución se ha implementado como una aplicación web basada en *Flask* [20], siguiendo una arquitectura modular que permite separar las distintas responsabilidades del sistema. Internamente la aplicación se encuentra organizada en varios módulos que facilitan su mantenimiento y evolución, distribuyendo las distintas funcionalidades del sistema en componentes independientes.

Por un lado, la aplicación incorpora una capa de presentación encargada de gestionar la interacción con el usuario mediante una interfaz web accesible desde el navegador. A través de ella, es posible administrar la información necesaria para la planificación académica, configurar los parámetros de generación de horarios e iniciar el proceso de generación.

La gestión de la información se realiza mediante archivos en formato *JSON*, utilizados para almacenar de forma estructurada los datos académicos necesarios para el funcionamiento del sistema. Esta solución permite simplificar la arquitectura de la aplicación y evita la necesidad de incorporar un sistema gestor de bases de datos.

El núcleo funcional de la aplicación está constituido por el generador de horarios, implementado como un módulo independiente encargado de construir soluciones válidas a partir de los datos disponibles y las restricciones definidas. De forma complementaria, las restricciones blandas se encuentran desacopladas del generador principal e implementadas en un módulo independiente, lo que facilita su mantenimiento y permite incorporar nuevos criterios de evaluación sin modificar la lógica principal del algoritmo.

En cuanto a la interfaz de usuario, la aplicación integra la librería *Bootstrap* [21] mediante CDN, lo que permite agilizar el desarrollo del diseño visual y garantizar una interfaz responsiva y adaptable a distintos dispositivos.

Esta organización modular favorece la separación de responsabilidades entre los distintos componentes del sistema, mejorando su legibilidad, mantenibilidad y capacidad de ampliación futura.

4.2.2 Modelo de Datos

El modelo de datos define la estructura utilizada por la aplicación para representar la información necesaria en el correcto funcionamiento del algoritmo. Dado que el sistema no hace uso de una base de datos, toda la información se almacena en un archivo en formato *JSON*, que actúa como repositorio central de datos.

El uso de este formato permite una estructura flexible y fácilmente modificable, facilitando la incorporación de nuevos elementos o la actualización de los existentes sin necesidad de modificar la lógica interna del sistema.

El diseño general del modelo de datos se encuentra inspirado en las estructuras de representación utilizadas en diferentes ediciones de la *International Timetabling Competition* (ITC) [22], una de las principales referencias académicas en el ámbito de la generación automática de horarios. En particular, se adopta la separación entre entidades fundamentales del problema, como asignaturas, docentes, aulas y franjas temporales, adaptando posteriormente dicha representación a las necesidades específicas del contexto universitario abordado en este proyecto.

No obstante, el modelo implementado no constituye una reproducción directa de ninguno de los formatos definidos por la ITC, sino una adaptación simplificada orientada a facilitar tanto la implementación del algoritmo como la gestión de los datos por parte de los usuarios de la aplicación.

4.2.2.1 Entidades Principales

El modelo de datos se compone de varias entidades que representan los elementos fundamentales del problema de planificación abordado.

Las principales entidades existentes en el modelo de datos son las siguientes:

- **Aulas** (*rooms*): Representan los espacios físicos disponibles. Esta entidad almacena el número de aula junto a su capacidad y edificio correspondiente.
- **Docentes** (*teachers*): Contiene la información relativa al profesorado disponible para la impartición de las asignaturas. Esta entidad almacena el nombre del docente, la facultad o facultades a las que pertenece y la disponibilidad horaria del mismo.

- **Cursos** (*courses*): Representa cada una de las asignaturas que deben ser planificadas por el algoritmo. Es la entidad central de la aplicación, la más relevante dentro de la aplicación y la que debe ser asignada por el algoritmo a un espacio de tiempo. Por tanto, explicaré cada atributo de forma individual.
 - **name**: Almacena el nombre completo de la asignatura.
 - **term**: Almacena el semestre en el que se imparte la asignatura.
 - **teacher**: Contiene el ID del docente, relacionado directamente con la entidad *teachers*.
 - **grades**: Contiene la lista de cursos que asisten a la asignatura.
 - **students**: El número de estudiantes matriculados en la asignatura.
 - **sessions_per_week**: Cantidad de sesiones que se imparten durante la semana.
 - **duration_slots**: Cantidad en *timeslots* de duración de cada sesión. Teniendo en cuenta que cada *timeslot* representa 30 minutos.
 - **possible_rooms**: Lista de aulas en las que podrá impartirse la asignatura.
 - **optativa**: Un *boolean* que indica si una asignatura es optativa u obligatoria.

4.2.2.2 Configuración

El archivo JSON tiene al comienzo una sección "*config*" que contiene parámetros relevantes para la asignación de cursos a espacios de tiempo.

Este apartado de configuración contiene los siguientes tres campos:

- **timeslots_per_day**: Contiene la cantidad de *timeslots* en la que se dividen las 24 horas de un día.
- **minutes_per_slot**: Contiene la duración de cada *slot* individual en minutos.
- **start_hour**: Indica la hora a partir de la que el algoritmo podrá comenzar a asignar sesiones.

Para abordar el caso, estos últimos campos son fundamentales, sobre todo la división del tiempo en *timeslots*, ya que esto nos permite definir franjas de tiempo de una forma organizada y fácil de comprender para el algoritmo.

En este caso, el día está dividido en 48 *timeslots*, representando cada uno 30 minutos de la siguiente manera:

- *timeslot* 1 → 00:00 - 00:30
- *timeslot* 2 → 00:30 - 01:00
- *timeslot* 3 → 01:00 - 01:30
- ...
- *timeslot* 48 → 23:30 - 00:00

4.2.3 Diseño del Algoritmo

El objetivo principal del algoritmo desarrollado es generar automáticamente horarios académicos válidos a partir de la información almacenada en el sistema. Para ello, debe asignar cada asignatura a un conjunto de franjas horarias y aulas disponibles, garantizando el cumplimiento de las restricciones obligatorias y optimizando posteriormente la calidad de las soluciones obtenidas.

La generación de horarios constituye un problema combinatorio con un elevado número de posibles soluciones. A medida que aumenta el número de asignaturas, docentes, aulas y restricciones, el espacio de búsqueda crece rápidamente, haciendo inviable la exploración exhaustiva de todas las combinaciones posibles. Por este motivo, se ha optado por implementar un algoritmo heurístico propio capaz de construir soluciones válidas en tiempos razonables y mejorarlas posteriormente mediante un proceso de optimización.

El algoritmo se encuentra dividido en dos fases principales. En primer lugar, se realiza la construcción de soluciones válidas respetando las restricciones consideradas obligatorias. Posteriormente, las soluciones generadas son evaluadas y optimizadas mediante un sistema de penalizaciones asociado a las restricciones blandas definidas por el usuario.

4.2.3.1 Restricciones Duras

Las restricciones duras representan aquellas condiciones cuyo incumplimiento invalida completamente un horario. Durante el proceso de generación, cada nueva asignación propuesta es verificada antes de incorporarse a la solución, garantizando que todas las restricciones obligatorias se satisfacen en todo momento.

Las restricciones consideradas por el algoritmo son las siguientes:

- Una misma aula no puede albergar dos clases simultáneamente.
- Un docente no puede impartir más de una clase al mismo tiempo.
- Un mismo grupo académico no puede asistir a dos clases simultáneamente.
- Una asignatura no puede impartirse más de una vez durante el mismo día.
- Un aula no puede contener un número de estudiantes superior a su capacidad.

Para comprobar estas restricciones, el algoritmo analiza los posibles solapamientos temporales entre las sesiones ya asignadas y la nueva candidata. Únicamente aquellas asignaciones que cumplen todas las restricciones son incorporadas al horario en construcción.

4.2.3.2 Restricciones Blandas

Además de las restricciones obligatorias, el sistema incorpora un conjunto de restricciones blandas cuyo objetivo es evaluar la calidad de los horarios generados. A diferencia de las restricciones duras, su incumplimiento no invalida una solución, sino que incrementa una penalización asociada a la misma.

Cada restricción blanda se implementa de forma independiente mediante una función específica encargada de analizar el horario generado y devolver una penalización numérica. La puntuación total de una solución se obtiene como la suma de todas las penalizaciones aplicables, de manera que cuanto menor sea la puntuación obtenida, mayor será la calidad del horario.

Este diseño modular permite incorporar nuevos criterios de evaluación sin necesidad de modificar la lógica principal del algoritmo, facilitando su mantenimiento y ampliación futura.

Las restricciones blandas implementadas actualmente son las siguientes:

- Evitar días con una única clase para un mismo grupo académico.
- Equilibrar la distribución de clases a lo largo de la semana.
- Minimizar los huecos entre clases consecutivas.
- Reducir los desplazamientos entre edificios.
- Penalizar asignaciones que coincidan con periodos de indisponibilidad del profesorado.

Cada una de estas restricciones posee un peso específico dentro del sistema de penalización, de manera que aquellas consideradas más perjudiciales generan una penalización superior. Entre ellas destaca la restricción asociada a la indisponibilidad del profesorado, cuyo incumplimiento recibe la penalización más elevada.

Asimismo, el sistema permite al usuario seleccionar qué restricciones blandas desea considerar durante el proceso de generación. De este modo, únicamente las restricciones activadas son empleadas para evaluar y comparar la calidad de los horarios obtenidos.

4.2.3.3 Proceso de Generación de Horarios

La construcción de soluciones se realiza mediante un procedimiento recursivo de búsqueda con retroceso (*backtracking*) [23]. El algoritmo procesa secuencialmente las asignaturas que deben ser planificadas e intenta asignar cada una de sus sesiones a una combinación válida de día, franja horaria y aula.

Para cada sesión, el algoritmo explora las distintas alternativas disponibles y verifica el cumplimiento de las restricciones duras definidas. Cuando una asignación resulta válida, esta se incorpora temporalmente a la solución parcial y el proceso continúa con la siguiente sesión. En caso de que no sea posible completar el horario a partir de una determinada asignación, el algoritmo retrocede y prueba una alternativa diferente.

Con el objetivo de aumentar la diversidad de soluciones generadas, el orden de procesamiento de las asignaturas y de las alternativas consideradas durante la búsqueda se seleccionan de forma aleatoria en cada ejecución. Esto permite explorar distintas regiones del espacio de búsqueda y obtener horarios diferentes en sucesivos intentos, como se muestra en la figura 4.2.

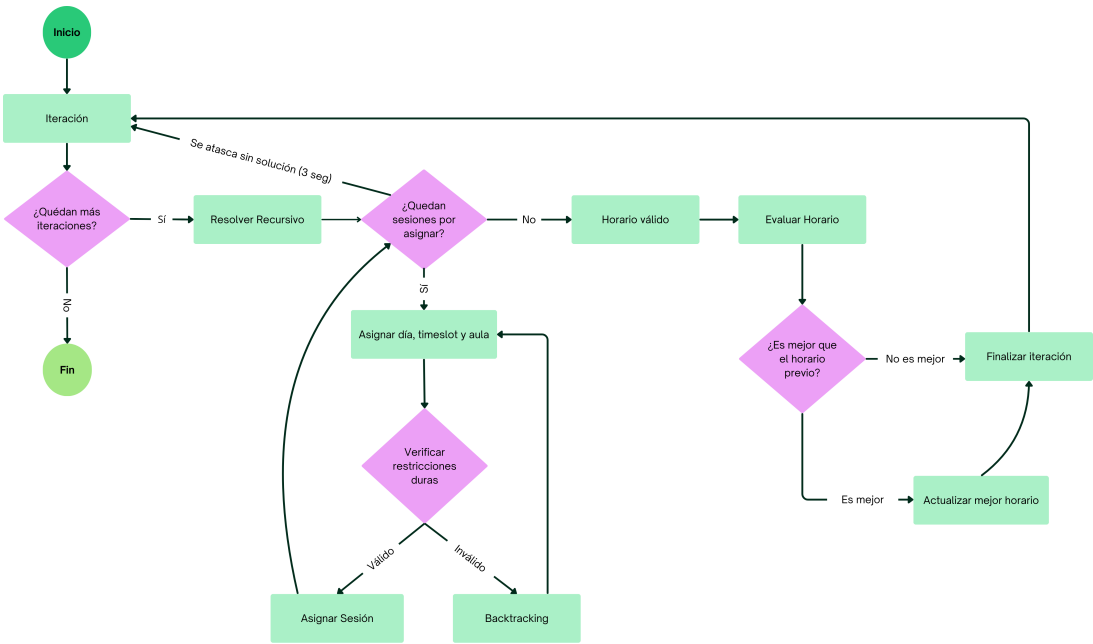


Figura 4.1: Diagrama de flujo de la fase de generación de horarios

Hora	Lunes	Martes	Miércoles	Jueves	Viernes
08:00					
08:30	Administración de empresas (ES) 41 (Ed-Arturo) González-Pernía, José Luis		Electrónica Digital (ES) 40 (Ed-Arturo) Barro Planells, Alex		
09:00					
09:30					
10:00					
10:30	Programación I (ES) 42 (Ed-Arturo) García Góngoli, Ander	Álgebra (ES) 42 (Ed-Arturo) Diez Blanco, Luis Enrique	Administración de empresas (ES) 41 (Ed-Arturo) González-Pernía, José Luis	Electrónica Digital (ES) 40 (Ed-Arturo) Barro Planells, Alex	Introducción a los computadores (ES) 43 (Ed-Arturo) Barro Sánchez, Fernando
11:00					
11:30					
12:00					
12:30		Introducción a los computadores (ES) 40 (Ed-Arturo) Barro Sánchez, Fernando	Álgebra (ES) 42 (Ed-Arturo) Diez Blanco, Luis Enrique	Álgebra (ES) 42 (Ed-Arturo) Diez Blanco, Luis Enrique	Programación I (ES) 27 (Ed-Enardonea) García Góngoli, Ander
13:00					
13:30					

Hora	Lunes	Martes	Miércoles	Jueves	Viernes
08:00					
08:30	Introducción a los computadores (ES) 40 (Ed-Arturo) Barro Sánchez, Fernando		Introducción a los computadores (ES) 40 (Ed-Arturo) Barro Sánchez, Fernando		Administración de empresas (ES) 41 (Ed-Arturo) González-Pernía, José Luis
09:00					
09:30					
10:00					
10:30	Electrónica Digital (ES) 41 (Ed-Arturo) Barro Planells, Alex	Álgebra (ES) 43 (Ed-Arturo) Diez Blanco, Luis Enrique	Electrónica Digital (ES) 42 (Ed-Arturo) Barro Planells, Alex		Programación I (ES) 28 (Ed-Enardonea) García Góngoli, Ander
11:00					
11:30					
12:00					
12:30	Administración de empresas (ES) 40 (Ed-Arturo) González-Pernía, José Luis	Programación I (ES) 28 (Ed-Enardonea) García Góngoli, Ander	Álgebra (ES) 42 (Ed-Arturo) Diez Blanco, Luis Enrique		Álgebra (ES) 40 (Ed-Arturo) Diez Blanco, Luis Enrique
13:00					
13:30					

Figura 4.2: Comparación de distintas soluciones generadas por el algoritmo (1º Ingeniería Informática)

4.2.3.4 Generación de Múltiples Soluciones

La aplicación no se limita a generar una única solución, sino que realiza múltiples intentos independientes de construcción de horarios. En cada ejecución se generan hasta veinte soluciones candidatas, partiendo de distintos órdenes de exploración producidos mediante aleatorización.

Con el fin de evitar tiempo de ejecución excesivos en situaciones donde la búsqueda queda bloqueada, cada intento dispone de un límite máximo de tiempo. Si durante ese periodo no se consigue completar una solución válida, el intento se descarta y el algoritmo continúa con el siguiente.

Todas las soluciones válidas obtenidas son evaluadas mediante el sistema de restricciones blandas descrito anteriormente. La calidad de cada horario se determina a partir de la puntuación de penalización obtenida, conservándose aquellas soluciones que presentan una mejor valoración para su posterior optimización.

4.2.3.5 Optimización de Soluciones

Una vez obtenida una solución válida que satisface todas las restricciones duras, el algoritmo aplica una fase adicional de optimización cuyo objetivo es mejorar la calidad del horario generado. Esta etapa se centra exclusivamente en reducir las penalizaciones asociadas a las restricciones blandas, manteniendo en todo momento la validez de la solución.

El proceso de optimización parte del mejor horario obtenido durante la fase de generación. A continuación, se realizan múltiples iteraciones en las que se selecciona aleatoriamente una de las sesiones asignadas y se propone una nueva ubicación temporal para ella, modificando su día o franja horaria.

Cada modificación propuesta es evaluada antes de ser aceptada. En primer lugar, se verifica que la nueva asignación continúa cumpliendo todas las restricciones duras definidas por el sistema. Posteriormente, se calcula la puntuación total del horario teniendo en cuenta las restricciones blandas activadas.

Si la nueva configuración produce una reducción de la penalización total, la modificación es aceptada y pasa a formar parte de la solución actual. En caso contrario, la propuesta es descartada y el algoritmo conserva la mejor solución conocida hasta ese momento.

Este procedimiento se repite durante un número fijo de iteraciones, permitiendo explorar distintas configuraciones del horario y corregir progresivamente situaciones poco deseables, como la aparición de huecos entre clases, distribuciones desequilibradas de la carga lectiva o cambios innecesarios de edificio.

La estrategia empleada puede considerarse una búsqueda local basada en mejoras sucesivas, ya que parte de una solución válida previamente construida y realiza pequeñas modificaciones orientadas a reducir su coste. Aunque no garantiza la obtención de la solución óptima global, permite mejorar significativamente la calidad de los horarios generados manteniendo unos tiempos de ejecución reducidos.

```
--- Iniciando Optimización ---  
Puntuación inicial: 38000  
Puntuación tras 1400 iteraciones: 18500  
Puntuación tras 2100 iteraciones: 18000  
Puntuación tras 2200 iteraciones: 18000  
Puntuación tras 2600 iteraciones: 18000  
Puntuación tras 3100 iteraciones: 11500  
Optimización finalizada. Puntuación final: 10000  
Mejora total: 28000 puntos.  
-----
```

Figura 4.3: Logs por consola obtenidos durante el proceso de optimización

4.2.4 Diseño de la Interfaz

Además de la generación automática de horarios, uno de los objetivos principales del proyecto ha sido proporcionar una herramienta sencilla e intuitiva para los responsables de la planificación académica. Por este motivo, se ha prestado especial atención al diseño de la interfaz de usuario, buscando facilitar tanto la gestión de la información como la consulta de los horarios generados.

La aplicación presenta una interfaz web, desarrollada utilizando tecnologías estándar como HTML, CSS y JavaScript. Asimismo, se ha empleado la librería Bootstrap para simplificar el diseño visual y garantizar una correcta adaptación a diferentes tamaños de pantalla.

Durante el diseño de la interfaz se han seguido varios criterios fundamentales. En primer lugar, se ha buscado mantener una estructura clara y coherente en todas las páginas de la aplicación, facilitando la navegación entre las distintas funcionalidades disponibles. En segundo lugar, se ha procurado reducir al máximo la complejidad de las tareas de administración de datos mediante formularios sencillos y mecanismos de edición intuitivos. Finalmente, se ha prestado especial atención a la representación visual de los horarios generados, permitiendo interpretar fácilmente la información producida por el algoritmo.

4.3 CONSIDERACIONES SOBRE LA IMPLEMENTACIÓN

Una vez definidas las decisiones de diseño adoptadas para el desarrollo del sistema, resulta necesario describir los aspectos más relevantes relacionados con su implementación. En esta sección se presentan las tecnologías

empleadas, la organización interna del proyecto y diversas consideraciones técnicas que han condicionado el desarrollo de la aplicación.

El objetivo de este apartado no es profundizar en el funcionamiento del algoritmo o en el diseño de la interfaz, aspectos ya abordados en las secciones anteriores, sino describir cómo dichas decisiones se han materializado en una solución software funcional. Asimismo, se detallan algunos aspectos relacionados con la estructura del proyecto, la gestión de los datos y la organización del código fuente, con el fin de proporcionar una visión más completa de la implementación realizada.

4.3.1 Tecnologías Utilizadas

El desarrollo de la aplicación se ha realizado empleando un conjunto de tecnologías orientadas a la construcción de aplicaciones web ligeras, priorizando la simplicidad de desarrollo, la mantenibilidad del código y la facilidad de despliegue.

El lenguaje principal utilizado ha sido *Python*, elegido por su sintaxis clara, su amplio ecosistema de librerías y su idoneidad para la implementación de algoritmos de optimización y resolución de problemas combinatorios. Además, su flexibilidad ha permitido integrar de forma sencilla tanto la lógica de negocio de la aplicación como el algoritmo de generación de horarios.

Para el desarrollo de la aplicación web se ha utilizado el framework *Flask*. Esta herramienta proporciona una estructura ligera y flexible para la construcción de aplicaciones web, permitiendo integrar de manera sencilla la lógica del sistema con la interfaz de usuario. Su carácter minimalista ha resultado especialmente adecuado para las necesidades del proyecto, evitando incorporar componentes innecesarios y facilitando el control sobre la organización interna de la aplicación.

La interfaz de usuario se ha desarrollado mediante las tecnologías estándar de la Web: HTML para la definición de la estructura de las páginas, CSS para la personalización de la presentación visual y JavaScript para la implementación de funcionalidades dinámicas e interactivas en el navegador.

Con el objetivo de agilizar el desarrollo de la interfaz y garantizar una experiencia de usuario consistente, se ha empleado la librería *Bootstrap* [21]. Esta proporciona un conjunto de componentes visuales reutilizables y un sistema de diseño responsivo que permite adaptar la aplicación a diferentes tamaños de pantalla sin necesidad de desarrollar soluciones específicas para cada dispositivo.

Por último, la gestión de dependencias del proyecto se ha realizado mediante *Poetry*, una herramienta que faci-

lita la administración de paquetes y la creación de entornos virtuales aislados. Su utilización permite garantizar la reproducibilidad del entorno de desarrollo y simplifica la instalación de la aplicación en distintos equipos.

4.3.2 Estructura del Proyecto

Con el objetivo de facilitar el mantenimiento y la evolución de la aplicación, el código fuente se ha organizado siguiendo una estructura modular que separa las distintas responsabilidades del sistema. Esta organización permite localizar fácilmente cada componente, favorece la reutilización de código y simplifica la incorporación de nuevas funcionalidades.

La aplicación se encuentra estructurada en varios directorios principales, cada uno de ellos orientado a una función específica dentro del sistema. La Figura 4.4 muestra una representación simplificada de la organización general del proyecto.

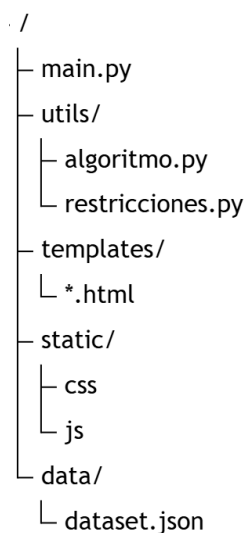


Figura 4.4: Vista de árbol del proyecto

El archivo *main.py* constituye el punto de entrada principal de la aplicación y contiene la configuración del servidor web, así como la definición de las rutas encargadas de gestionar las distintas funcionalidades accesibles desde la interfaz.

El directorio *utils* agrupa los componentes relacionados con la lógica interna del sistema. En particular, el módulo *algoritmos.py* implementa el proceso de generación y optimización de horarios, mientras que *restricciones.py* contiene las distintas restricciones blandas utilizadas para evaluar la calidad de las soluciones generadas.

Las plantillas HTML utilizadas para consturir las páginas de la aplicación se encuentran almacenadas en el directorio *templates*. Flask emplea estas plantillas para generar dinámicamente la interfaz mostrada al usuario en función de la información disponible en cada momento.

Por su parte, el directorio *static* contiene los recursos estáticos necesarios para la interfaz de usuario, incluyendo la hoja de estilo CSS y archivos JavaScript utilizados para complementar el comportamiento de las páginas web.

Finalmente, el directorio *data* almacena el archivo JSON empleado como repositorio de datos de la aplicación. En él se conserva toda la información académica necesaria para la generación de horarios, incluyendo asignaturas, docentes, aulas, grupos y parámetros de configuración.

4.3.3 Elección de Sistema de Almacenamiento

Una de las decisiones adoptadas durante el desarrollo del proyecto fue utilizar un archivo en formato *JSON* como mecanismo principal de almacenamiento de información, en lugar de emplear un sistema gestor de bases de datos relacional. Esta elección se realizó teniendo en cuenta las características y necesidades específicas de la aplicación desarrollada.

En primer lugar, el uso de un único archivo JSON permite simplificar significativamente la arquitectura del sistema. Al no requerir la instalación y configuración de una bases de datos, se reduce la complejidad de despliegue de la aplicación y se facilita su ejecución en distintos entornos. Esta solución resulta especialmente adecuada para aplicaciones de tamaño reducido o medio, como la desarrollada en este proyecto.

Otro aspecto relevante es el soporte natural que JSON ofrece para la representación de estructuras de datos complejas y jerárquicas. Mediante el uso de objetos anidados y listas, es posible modelar de forma directa información como los grupos académicos asociados a una asignatura, las aulas disponibles para su impartición o las franjas de indisponibilidad del profesorado. Esta capacidad permite representar relaciones complejas de manera intuitiva y cercana a las estructuras utilizadas internamente por la aplicación.

Por último, JSON constituye uno de los formatos de intercambio de información más extendidos en la actualidad y cuenta con soporte nativo en prácticamente todos los lenguajes y tecnologías web modernas. Esta amplia compatibilidad facilita la integración con otros sistemas, simplifica el procesamiento de los datos y favorece la portabilidad futura de la aplicación.

Considerando estos factores, la utilización de un archivo JSON se consideró la alternativa más adecuada para las necesidades del proyecto, proporcionando una solución sencilla, flexible y suficientemente eficiente para

gestionar la información académica empleada por el generador de horarios.

Característica	JSON	MySQL	PostgreSQL
Facilidad de despliegue	Muy alta	Media	Media
Requiere instalación de servidor	No	Sí	Sí
Configuración inicial	Mínima	Moderada	Moderada
Flexibilidad para modificar la estructura de datos	Muy alta	Media	Media
Adecuado para estructuras jerárquicas	Muy alta	Media	Alta
Coste de mantenimiento	Muy bajo	Medio	Medio
Escalabilidad para grandes volúmenes de datos	Baja	Alta	Muy alta
Consultas complejas y optimización	Baja	Alta	Muy alta
Rendimiento en aplicaciones pequeñas	Alto	Alto	Alto
Portabilidad de los datos	Muy alta	Media	Media

Cuadro 4.1: Comparación entre distintas alternativas de almacenamiento

Como puede observarse en la Tabla 4.1, las bases de datos relacionales como MySQL y PostgreSQL ofrecen mayores capacidades de escalabilidad, optimización de consultas e integridad de los datos. Sin embargo, estas ventajas no resultan determinantes para una aplicación de tamaño reducido como la desarrollada en este proyecto.

Por el contrario, JSON proporciona una solución más sencilla de desplegar y mantener, evitando la necesidad de instalar y administrar un sistema gestor de bases de datos. Además, su flexibilidad para representar estructuras jerárquicas y modificar el esquema de los datos durante el desarrollo facilitó la implementación de las entidades necesarias para el correcto funcionamiento del sistema. Por estos motivos, se consideró la alternativa más adecuada para los requisitos y la escala de la aplicación desarrollada.

4.3.4 Estrategia de Generación

La generación automática de horarios supone un problema ampliamente estudiado dentro del ámbito de la investigación organizativa y la optimización combinatoria. Este tipo de problemas pertenece a la categoría conocida como *Automated Timetabling* [1] o *Timetabling Problem*, cuyo objetivo consiste en asignar recursos limitados —como aulas, docentes y franjas horarias— satisfaciendo un conjunto de restricciones obligatorias y

preferencias adicionales.

Desde el punto de vista computacional, el problema del *timetabling* está clasificado como un problema *NP-hard*. Esta clasificación implica que, hasta la fecha, no se conoce ningún algoritmo capaz de encontrar de forma eficiente la solución óptima para todos los casos posibles. A medida que aumentan el número de asignaturas, grupos, docentes y restricciones, el número de combinaciones posibles crece de forma exponencial, haciendo inviable la exploración exhaustiva de todas las alternativas en tiempos razonables.

Como consecuencia, se proponen múltiples enfoques para abordar este tipo de problemas, incluyendo técnicas exactas, metaheurísticas y algoritmos híbridos. Sin embargo, no existe una solución universalmente aceptada como la mejor para todos los escenarios, ya que la eficacia de cada método depende en gran medida de las características concretas del problema considerado, así como de las restricciones y objetivos definidos.

Por este motivo, en lugar de adoptar una solución específica, se optó por desarrollar una estrategia propia adaptada a los requisitos de la Universidad de Deusto y a las necesidades funcionales de la aplicación desarrollada. La propuesta implementada combina una fase de generación basada en búsqueda con retroceso (*backtracking*) y una fase posterior de optimización mediante criterios heurísticos.

La utilización de *backtracking* permite garantizar el cumplimiento de las restricciones consideradas obligatorias durante el proceso de construcción de los horarios, descartando de forma temprana aquellas asignaciones que conducen a estados inviables. Posteriormente, las soluciones válidas obtenidas son evaluadas mediante un conjunto de restricciones blandas que permiten estimar su calidad desde el punto de vista organizativo.

Asimismo, se decidió desacoplar la generación de soluciones de la evaluación de calidad mediante restricciones blandas. Esta separación facilita la incorporación de nuevos criterios de optimización y mejora la mantenibilidad del sistema, permitiendo adaptar el comportamiento del algoritmo a futuras necesidades sin modificar el núcleo del proceso de generación.

En conjunto, esta aproximación proporciona un equilibrio adecuado entre calidad de las soluciones, flexibilidad de implementación y tiempo de ejecución, características especialmente relevantes para un proyecto con los requisitos y alcance definidos en este trabajo.

Por último, es importante señalar que, aunque la estrategia desarrollada fue diseñada teniendo en cuenta las necesidades y requisitos identificados para la Universidad de Deusto, las limitaciones temporales y de recursos inherentes al desarrollo de un Trabajo Fin de Grado impidieron abordar todas las funcionalidades y mejoras potenciales que podrían incorporarse a una solución de carácter plenamente institucional.

En consecuencia, la aplicación desarrollada debe entenderse como una prueba de concepto funcional capaz de resolver de forma satisfactoria el problema planteado dentro del alcance definido para este proyecto. No obstante, existen diversas líneas de mejora relacionadas tanto con la ampliación de las restricciones consideradas como con la incorporación de elementos o situaciones más complejas, aspectos que se presentan y discuten en el apartado de trabajo futuro.

4.3.5 Implementación de Restricciones

Uno de los aspectos fundamentales de la aplicación desarrollada es la gestión de las restricciones que condicionan la generación de horarios. Dado que el número y la naturaleza de estas restricciones pueden evolucionar con el tiempo, durante el desarrollo se consideró prioritario diseñar una solución flexible y fácilmente extensible.

Con este objetivo, las restricciones fueron implementadas de forma modular e independiente del núcleo principal del algoritmo. En lugar de concentrar toda la lógica de validación en una única función, cada restricción se encapsula en componentes específicos encargados de evaluar una condición concreta. Esta aproximación reduce el acoplamiento entre las distintas partes del sistema y facilita tanto el mantenimiento como la incorporación de nuevas restricciones en futuras versiones de la aplicación.

El fragmento de código 4.1 muestra una parte del diccionario de restricciones blandas, utilizado para llamar a las funciones correspondientes cuando el usuario las selecciona a la hora de generar un horario, además de incluir una descripción asociada que se muestra mediante la interfaz para comprender que hace cada restricción.

```

1  # Diccionario para mapear los IDs que envíe el usuario con las funciones
2  RESTRICCIONES_DISPONIBLES = {
3      "evitar_clase_unica": {
4          "func": penalizar_clase_unica,
5          "label": "Clase Unica",
6          "description": "Evita que un grado tenga un unico bloque de clase en
7                          un dia."
8      },
9      "equilibrar_tres_clases": {
10         "func": penalizar_tres_clases,
11         "label": "Tres Clases",
12         "description": "Busca evitar que un grado tenga exactamente tres
13                         clases en el mismo dia."
14     }
15  }
```

```

12     }
13 }

```

Seudocódigo 4.1: Diccionario de restricciones que se importa en el algoritmo

La separación entre restricciones duras y restricciones blandas también se trasladó a la implementación. Las restricciones duras son verificadas durante el proceso de generación de horarios y actúan como criterios de validez. Cuando una asignación incumple alguna de estas condiciones, la solución parcial es descartada inmediatamente, evitando explorar combinaciones inviables y reduciendo el espacio de búsqueda. Esta integración responde al hecho de que dichas restricciones representan condiciones indispensables para la validez de cualquier horario generado. Su incumplimiento produce soluciones inviables desde el punto de vista académico u organizativo, por lo que deben verificarse durante la construcción de la solución y no únicamente durante su evaluación posterior.

Por otro lado, las restricciones blandas no determinan la validez de un horario, sino que permiten evaluar su calidad. Su implementación se basa en un sistema de penalizaciones mediante el cual cada incumplimiento añade un determinado coste a la solución. De esta manera, el algoritmo puede comparar distintos horarios válidos y priorizar aquellos que presentan una menor penalización global.

Un caso especialmente relevante es la restricción relacionada con la disponibilidad del profesorado. Desde el punto de vista organizativo, existen situaciones en las que un docente preferiría evitar impartir docencia en determinadas franjas horarias —como impartir a las ocho de la mañana si se viene desde Bilbao— y otras en las que dicha disponibilidad resulta prácticamente imposible. No obstante, incorporar todas estas limitaciones como restricciones duras habría reducido significativamente el espacio de soluciones factibles, dificultando la obtención de horarios válidos y aumentando considerablemente el tiempo de búsqueda necesario para completar el proceso de generación.

Por este motivo, se optó por modelar la disponibilidad del profesorado como una restricción blanda de alta prioridad. De esta forma, el algoritmo continúa siendo capaz de generar soluciones incluso cuando no es posible satisfacer todas las preferencias establecidas, pero penaliza de manera significativa aquellas asignaciones que incumplen las franjas horarias indicadas por los docentes. Esta decisión permite mantener un equilibrio razonable entre la calidad de los horarios obtenidos y la capacidad del sistema para encontrar soluciones viables en tiempos de ejecución aceptables.

La Tabla 4.2 recoge las penalizaciones asociadas a cada una de las restricciones blandas consideradas durante el proceso de evaluación de las soluciones.

Restricción	Penalización	Justificación
Disponibilidad del profesorado	Variable	Se considera la preferencia más importante, por tanto la que mayor penalización otorga. Existen tres situaciones distintas a tener en cuenta, cada una con una penalización distinta. 1. Si el docente no puede impartir a ninguna hora a lo largo del día: 5000 2. Si el docente no puede impartir en un rango específico de horas y una sesión o una parte de la sesión se imparte en dicho rango: $\text{minutos_solapamiento} \times 20$ 3. Si el docente no puede impartir en un rango específico de días pero ninguna sesión se imparte en dicho rango: 0
Huecos entre clases	1000	Una situación frustrante para los estudiantes, gran causa de inasistencia.
Día con una única clase	500	Molesto sobre todo para estudiantes que viven lejos, causa de falta a clases.
Días de tres clases	200	Restricción menor, más opcional que el resto, evita sobrecargar a los estudiantes con días con sesiones largas.
Cambio de edificio	200	Restricción menor, al no ser un campus excesivamente grande, no supone un problema significativo.

Cuadro 4.2: Penalizaciones asociadas a las restricciones blandas

Los valores asignados a cada penalización fueron definidos de forma empírica durante el desarrollo del proyecto. El objetivo no consistía en obtener una función de evaluación matemáticamente óptima, sino en reflejar de forma aproximada la importancia relativa de cada criterio dentro del contexto académico considerado. En consecuencia, las penalizaciones más elevadas se asociaron a restricciones cuyo incumplimiento tiene un impacto significativo sobre la calidad percibida del horario, mientras que las de menor valor corresponden a preferencias cuya influencia resulta más limitada.

Esta arquitectura proporciona una elevada flexibilidad, ya que permite modificar los criterios de evaluación,

ajustar las penalizaciones asociadas a cada restricción o incorporar nuevas preferencias sin necesidad de alterar el funcionamiento general del algoritmo de generación. Como consecuencia, la aplicación puede adaptarse con relativa facilidad a nuevos requisitos organizativos o normativas académicas que puedan surgir en el futuro.

4.3.6 Interfaz de Usuario

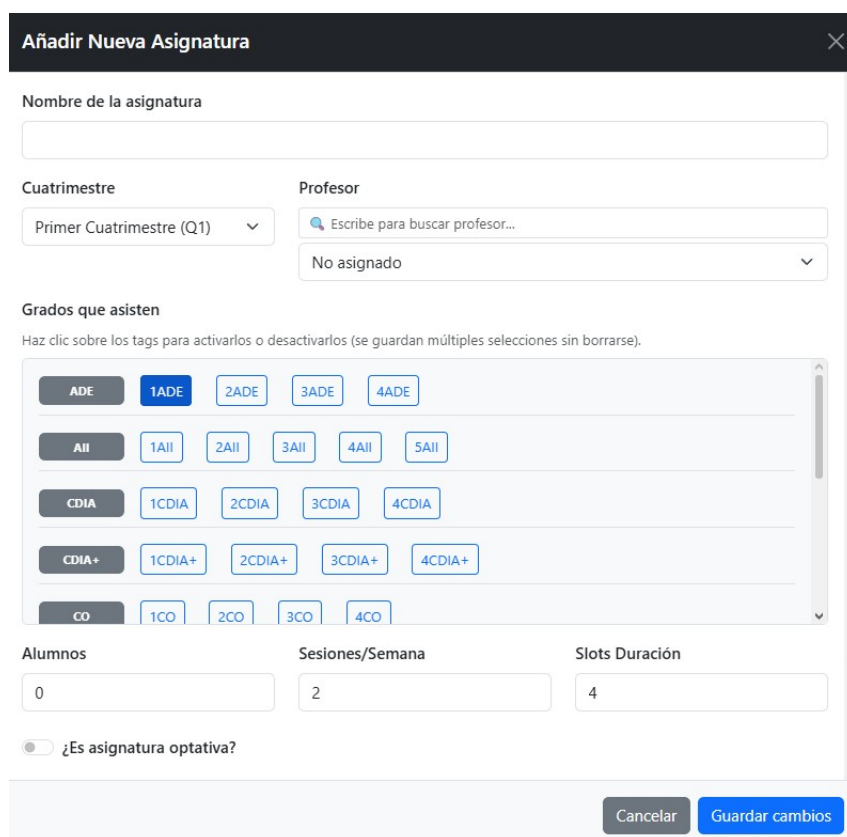
Aunque el objetivo principal del proyecto consiste en la generación automática de horarios académicos, la utilidad práctica de la aplicación depende en gran medida de la facilidad con la que los usuarios pueden gestionar la información necesaria para dicho proceso. Por este motivo, durante el desarrollo se prestó especial atención al diseño e implementación de una interfaz de usuario sencilla, intuitiva y eficiente.

La interfaz fue desarrollada utilizando tecnologías web estándar como HTML, CSS y JavaScript, complementadas mediante el framework Bootstrap. La elección de Bootstrap permitió acelerar el desarrollo de la aplicación gracias a la disponibilidad de componentes prediseñados y adaptables, garantizando además una apariencia visual homogénea en todas las secciones del sistema. Asimismo, su sistema de diseño facilita la creación de interfaces limpias y organizadas, reduciendo el esfuerzo necesario para implementar funcionalidades comunes de navegación y gestión de formularios.

La simplicidad de uso constituyó uno de los principales criterios de diseño. A diferencia de otros sistemas donde la configuración inicial permanece relativamente estable, la planificación académica requiere modificaciones frecuentes entre cursos académicos. Factores como la variación en el número de estudiantes matriculados, la incorporación o eliminación de asignaturas, los cambios en el profesorado o la disponibilidad de aulas obligan a actualizar periódicamente los datos utilizados para generar los horarios.

En consecuencia, la aplicación fue diseñada para que estas tareas de mantenimiento puedan realizarse de forma rápida y directa. La gestión de asignaturas, docentes, grupos académicos y aulas se lleva a cabo mediante formularios sencillos y mecanismos de edición accesibles, minimizando el número de pasos necesarios para realizar modificaciones sobre la información almacenada.

La Figura 4.5 muestra uno de los formularios utilizados para la gestión de la información académica. Este tipo de interfaces permite crear y modificar los datos necesarios para la generación de horarios de forma sencilla y centralizada.



Añadir Nueva Asignatura ✕

Nombre de la asignatura

Cuatrimestre
 Primer Cuatrimestre (Q1) ▼

Profesor

 No asignado ▼

Grados que asisten
 Haz clic sobre los tags para activarlos o desactivarlos (se guardan múltiples selecciones sin borrarse).

ADE	1ADE	2ADE	3ADE	4ADE	
AII	1AII	2AII	3AII	4AII	5AII
CDIA	1CDIA	2CDIA	3CDIA	4CDIA	
CDIA+	1CDIA+	2CDIA+	3CDIA+	4CDIA+	
CO	1CO	2CO	3CO	4CO	

Alumnos

Sesiones/Semana

Slots Duración

☐ ¿Es asignatura optativa?

Cancelar Guardar cambios

Figura 4.5: Formulario de creación de asignatura

Por otra parte, durante el diseño de la interfaz también se tuvieron en cuenta aquellas tareas que, debido a la cantidad de información involucrada, podían resultar especialmente repetitivas o costosas para el usuario. Entre ellas destacan la definición de las franjas horarias en las que un docente no se encuentra disponible o la selección de múltiples aulas candidatas para una misma asignatura.

Aunque estas operaciones son esenciales para el correcto funcionamiento del sistema y para la obtención de horarios de calidad, su realización manual mediante formularios convencionales podría resultar tediosa y consumir una cantidad considerable de tiempo, especialmente en escenarios con un elevado número de asignaturas, docentes o espacios disponibles.

Con el objetivo de minimizar este esfuerzo, se desarrollaron formularios específicos adaptados a este tipo de tareas, priorizando la rapidez de uso y la facilidad de comprensión. Estas interfaces permiten realizar selecciones múltiples de forma visual e intuitiva, reduciendo significativamente el tiempo necesario para introducir o modificar grandes volúmenes de información.

La Figura 4.6 muestra un ejemplo de este tipo de interfaz, utilizada para gestionar la disponibilidad horaria del profesorado mediante una representación visual que facilita la selección de franjas no disponibles.

Disponibilidad de Boto Sánchez, Fernando ✕

Selecciona los rangos horarios en los que el profesor NO está disponible:

☐ **Lunes**

08:00	08:30	09:00	09:30	10:00	10:30	11:00
11:30	12:00	12:30	13:00	13:30	14:00	14:30
15:00	15:30	16:00	16:30	17:00	17:30	18:00

☐ **Martes**

08:00	08:30	09:00	09:30	10:00	10:30	11:00
11:30	12:00	12:30	13:00	13:30	14:00	14:30
15:00	15:30	16:00	16:30	17:00	17:30	18:00

☐ **Miércoles**

08:00	08:30	09:00	09:30	10:00	10:30	11:00
11:30	12:00	12:30	13:00	13:30	14:00	14:30
15:00	15:30	16:00	16:30	17:00	17:30	18:00

☐ **Jueves**

08:00	08:30	09:00	09:30	10:00	10:30	11:00
11:30	12:00	12:30	13:00	13:30	14:00	14:30
15:00	15:30	16:00	16:30	17:00	17:30	18:00

☐ **Viernes**

08:00	08:30	09:00	09:30	10:00	10:30	11:00
11:30	12:00	12:30	13:00	13:30	14:00	14:30
15:00	15:30	16:00	16:30	17:00	17:30	18:00

Figura 4.6: Formulario de asignación de horas donde el docente no se encuentra disponible

Además, se buscó que la información relevante estuviese disponible de forma clara y fácilmente interpretable. Tanto los datos de configuración como los horarios generados se presentan mediante representaciones visuales estructuradas que permiten al usuario identificar rápidamente posibles incidencias y comprender el resultado obtenido por el algoritmo.

En conjunto, estas decisiones de diseño contribuyen a mejorar la experiencia de uso de la aplicación y reducen el esfuerzo necesario para mantener actualizada la información académica, aspecto especialmente importante en un entorno caracterizado por cambios recurrentes entre distintos cursos y semestres.

4.4 PLAN DE PRUEBAS

Con el objetivo de garantizar el correcto funcionamiento del sistema desarrollado, se ha definido un plan de pruebas orientado a verificar tanto la funcionalidad de la aplicación como la correcta generación de horarios bajo distintas condiciones de datos. La estrategia de pruebas ha evolucionado a lo largo del desarrollo del pro-

yecto, adaptándose a las distintas fases de madurez del sistema y al incremento progresivo de la complejidad del conjunto de datos.

En una primera fase, las pruebas se centraron en la verificación funcional de la aplicación. En este sentido, se comprobó el correcto funcionamiento de las operaciones básicas de gestión de datos, incluyendo la adición, modificación y eliminación de entidades como aulas, profesores, grados y asignaturas. Estas pruebas permitieron validar la consistencia de las interfaces de usuario y la correcta persistencia de la información en el sistema.

Posteriormente, durante las primeras iteraciones del desarrollo del algoritmo de generación de horarios, se utilizó un dataset reducido creado manualmente. Este conjunto de datos permitía realizar pruebas controladas sobre el comportamiento del algoritmo, facilitando la detección de errores y la verificación del cumplimiento de las restricciones definidas. En esta fase, la validación de resultados se realizó mediante inspección directa de los horarios generados.

Una vez implementada la fase de optimización del sistema, se introdujo un mecanismo de auditoría de restricciones, que permitió verificar de forma más sistemática el cumplimiento de las restricciones blandas definidas en el modelo. Esta auditoría, junto con el uso de logs detallados en consola, facilitó la depuración del algoritmo y la validación de su correcto funcionamiento en escenarios más complejos.

En fases posteriores del desarrollo, se incorporó la funcionalidad de importación de datos desde ficheros en formato .csv, lo que permitió aumentar significativamente el tamaño y realismo del conjunto de datos utilizado. Sin embargo, esta ampliación del dataset puso de manifiesto la presencia de errores e inconsistencias en los datos de entrada, lo que dificultaba la generación de horarios válidos por parte del sistema.

Como consecuencia, fue necesario llevar a cabo un proceso progresivo de depuración y limpieza del dataset, corrigiendo errores y refinando la estructura de los datos. Este proceso se realizó de forma iterativa, incorporando gradualmente cursos y datos ya validados al sistema. Para facilitar esta transición, se integró la funcionalidad de generación de horarios por cursos específicos, la cual permitió validar de manera incremental los nuevos datos depurados, asegurando su correcta integración en el sistema de planificación.

4.4.1 Casos de Prueba

Para complementar la estrategia de pruebas descrita anteriormente, se han definido una serie de casos de prueba representativos que permiten validar el comportamiento del sistema en distintas funcionalidades clave, documentando el resultado que se esperaba obtener y el resultado que realmente se obtuvo para cada una de las pruebas realizadas.

Caso de prueba	Entrada	Resultado esperado	Resultado obtenido
Alta de profesor	Creación de un profesor con nombre, disponibilidad y departamento	El profesor se almacena correctamente y aparece en la lista de gestión	Correcto
Modificación de aula	Cambio de capacidad de un aula existente	El sistema actualiza la capacidad sin afectar a otros registros	Correcto
Eliminación de asignatura	Eliminación de una asignatura vinculada a un grado	La asignatura desaparece del sistema y no se utiliza en la generación de horarios	Correcto
Generación de horario con dataset reducido	Dataset manual con 4 grados, 5 asignaturas por grado. Con grados que comparten asignaturas entre si	Se genera un horario sin conflictos de recursos ni solapamientos	Correcto
Validación de restricciones blandas	Generación de horario completo con varias restricciones blandas	El sistema minimiza la penalización aplicada mediante el proceso de optimización, mostrando los resultados constantemente mediante logs	Correcto
Importación CSV	Carga de ficheros con aulas, profesores y asignaturas facilitados por los gestores de la universidad	Los datos se importan correctamente en el sistema y están listos para usarse	Los datos de aulas y docentes no presentan ningún problema funcional. Los datos de asignaturas requieren de limpieza profunda
Validación de datos curados	Generación de horarios reducidos, que incrementan progresivamente a medida que se limpian datos	Los horarios se generan correctamente, siendo el algoritmo capaz de asignar las sesiones a franjas de tiempo cumpliendo las restricciones duras	Correcto, pero a medida que incrementa el tamaño del dataset el tiempo de espera y la probabilidad de no encontrar solución válida aumentan

Cuadro 4.3: Casos de prueba del sistema

5. MANUAL DE USUARIO

Este capítulo describe el funcionamiento de la aplicación desde la perspectiva del usuario final. Su objetivo es proporcionar una guía práctica para la utilización de las distintas funcionalidades implementadas, permitiendo gestionar la información necesaria para la generación de horarios académicos y consultar posteriormente los resultados obtenidos.

5.1 GESTIÓN DE DATOS

Antes de generar un horario es necesario registrar y configurar la información que utilizará el sistema durante el proceso de planificación. Para ello, la aplicación proporciona diferentes interfaces de gestión que permiten crear, modificar y eliminar los datos almacenados.

Las operaciones de gestión se organizan en tres áreas principales: aulas, docentes y asignaturas. Todas ellas presentan una estructura de funcionamiento similar, basada en formularios de edición y tablas de consulta que facilitan la visualización y modificación de la información existente.

Además, todas estas áreas cuentan con la posibilidad de importar datos mediante un fichero .csv, con funciones modeladas en base a la estructura utilizada por la universidad, no obstante, pese a que la importación tanto de aulas como de docentes no suponga ningún problema, la importación de grados y asignaturas genera un conjunto de datos que requieren de una posterior limpieza.

Cabe destacar que, siempre que se importen datos, independientemente de que sean aulas o docentes, deberá realizarse una verificación de que los datos para la generación se encuentran correctamente asignados.

5.1.1 Gestión de Aulas

En esta sección el usuario puede registrar las aulas disponibles para la impartición de clases. Para cada aula es posible definir características como su nombre identificativo, capacidad máxima y edificio al que pertenece.

Aulas

Importar Aulas (.csv)

Nueva Aula

ED-ALTUNA

40 Capacidad 70 pers. Editar Eliminar	41 Capacidad 87 pers. Editar Eliminar	42 Capacidad 85 pers. Editar Eliminar	43 Capacidad 82 pers. Editar Eliminar	44 Capacidad 72 pers. Editar Eliminar	45 Capacidad 68 pers. Editar Eliminar
46 Capacidad 50 pers. Editar Eliminar	47 Capacidad 48 pers. Editar Eliminar	48 Capacidad 47 pers. Editar Eliminar	49 Capacidad 46 pers. Editar Eliminar	Plato TV Capacidad 70 pers. Editar Eliminar	

ED-ARANZADI

B1 Capacidad 49 pers. Editar Eliminar	B2 Capacidad 20 pers. Editar Eliminar	B3 Capacidad 22 pers. Editar Eliminar	11 Capacidad 52 pers. Editar Eliminar	12 Capacidad 17 pers. Editar Eliminar	13 Capacidad 23 pers. Editar Eliminar
---	---	---	---	---	---

Figura 5.1: Ventana de gestión de aulas

5.1.2 Gestión de Docentes

Esta sección permite al usuario almacenar la información del profesorado y configurar aspectos relevantes para la generación de horarios. Entre ellos destaca la definición de franjas horarias no disponibles para el docente.

Esta página cuenta con una barra de búsqueda y una sección de filtros que permiten al usuario localizar docentes por nombre o por la facultad a la que pertenecen.

Por otra parte, todas las tarjetas de docente incorporan un menú desplegable que permite consultar las asignaturas que tiene asignadas. En caso de querer eliminar un docente, será necesario que no tenga ninguna asignatura asociada en el sistema.

Gestión de Docentes

Importar Docentes (.csv)

Nuevo Profesor

 Buscar profesor por nombre...

INGENIERÍA ▾

INGENIERÍA

Andrés Fernández, Alain T019 Asignaturas ▾ Disponibilidad Editar Eliminar	Madariaga Gómez, Aritz T020 Asignaturas ▾ Disponibilidad Editar Eliminar	Buján Carvallal, David T021 Asignaturas ▾ Disponibilidad Editar Eliminar
Díaz Labrador, Josuka T039 Asignaturas ▾ Disponibilidad Editar Eliminar	Lekuona Ezkurdia, Iñaki T053 Asignaturas ▾ Disponibilidad Editar Eliminar	Loyo Mendivil, Estibaliz T063 Asignaturas ▾ Disponibilidad Editar Eliminar
Boto Sánchez, Fernando T068 Asignaturas ▾ Disponibilidad Editar Eliminar Q1 • Introducción a los computadores (ES) • Programación III (ES) Q2 • Sistemas Inteligentes (ES) • Programación II (ES)	Quesada Granja, Carlos T070 Asignaturas ▾ Disponibilidad Editar Eliminar	Díez Blanco, Luís Enrique T075 Asignaturas ▾ Disponibilidad Editar Eliminar
	Juez Martín, Beatriz T086 Asignaturas ▾ Disponibilidad Editar Eliminar	Urdaneta Ponte, Cora T098 Asignaturas ▾ Disponibilidad Editar Eliminar

Figura 5.2: Ventana de gestión de docentes, filtrada en Ingeniería y mostrando el *dropdown* de asignaturas

5.1.3 Gestión de Grados y Asignaturas

Esta sección muestra las asignaturas que deberán aparecer en los horarios, cada una mostrada dentro de su grado, curso y cuatrimestre correspondiente. Para cada asignatura se especifican, entre otros datos, el docente responsable, el número de sesiones semanales y las aulas compatibles para su impartición.

Al igual que la ventana anterior, ofrece una barra de búsqueda que permite al usuario localizar una titulación por su nombre y navegar entre los distintos cursos de la misma mediante los botones situados en el propio recuadro del grado.

Plan de Estudios por Grado

Añadir Grado

Importar Asignaturas (.csv)

II Ingeniería Informática

III - Ingeniería Informática

Primer Cuatrimestre (Q1)

Administración de empresas (ES)

González-Pernía, José Luis

Sesiones: 2 | Horas Semanales: 4.0h | Alumnos: 52

Aulas: 40 41 42 43 45 46 48 49

Editar Aulas

Editar

Introducción a los computadores (ES)

Boto Sánchez, Fernando

Sesiones: 2 | Horas Semanales: 4.0h | Alumnos: 52

Aulas: 40 41 42 43 45 46 48 49

Editar Aulas

Editar

Electrónica Digital (ES)

Barco Martelo, Alex

Sesiones: 2 | Horas Semanales: 4.0h | Alumnos: 50

Aulas: 40 41 42 43 45 46 48 49 Sala ordenadores / ordenagailu gela

Editar Aulas

Editar

Segundo Cuatrimestre (Q2)

Cálculo (ES)

Díez Blanco, Luis Enrique

Sesiones: 2 | Horas Semanales: 4.0h | Alumnos: 50

Aulas: 43 42 40 49 46 41 45 48

Editar Aulas

Editar

Matemática Discreta (ES)

Aguilera Paz, Evelyn

Sesiones: 2 | Horas Semanales: 4.0h | Alumnos: 50

Aulas: 43 42 40 49 46 41 45 48

Editar Aulas

Editar

Física (ES)

García Zubia, Javier

Sesiones: 2 | Horas Semanales: 4.0h | Alumnos: 50

Aulas: 43 42 40 49 46 41 45 48

Editar Aulas

Editar

Añadir Asignatura

Eliminar Grado

Figura 5.3: Ventana de gestión de grados y asignaturas, filtrada en Ingeniería Informática

5.2 GENERACIÓN Y VISUALIZACIÓN DE HORARIOS

Una vez completada la configuración de los datos necesarios, el usuario puede iniciar el proceso de generación automática de horarios. Desde esta interfaz es posible seleccionar el semestre académico correspondiente y configurar las restricciones blandas que se desean considerar durante la búsqueda de soluciones.

Panel de Control del Algoritmo

Se ejecutarán 20 intentos. La vista final se organizará por Carrera/Grado.

Seleccionar Cuatrimestre: Primer Cuatrimestre (Q1) Selección de Grados Todos

Restricciones Opcionales

Activa los criterios blandos que quieras priorizar en el horario.

Tres Clases

Clase Única

Disponibilidad Profesores

Movilización

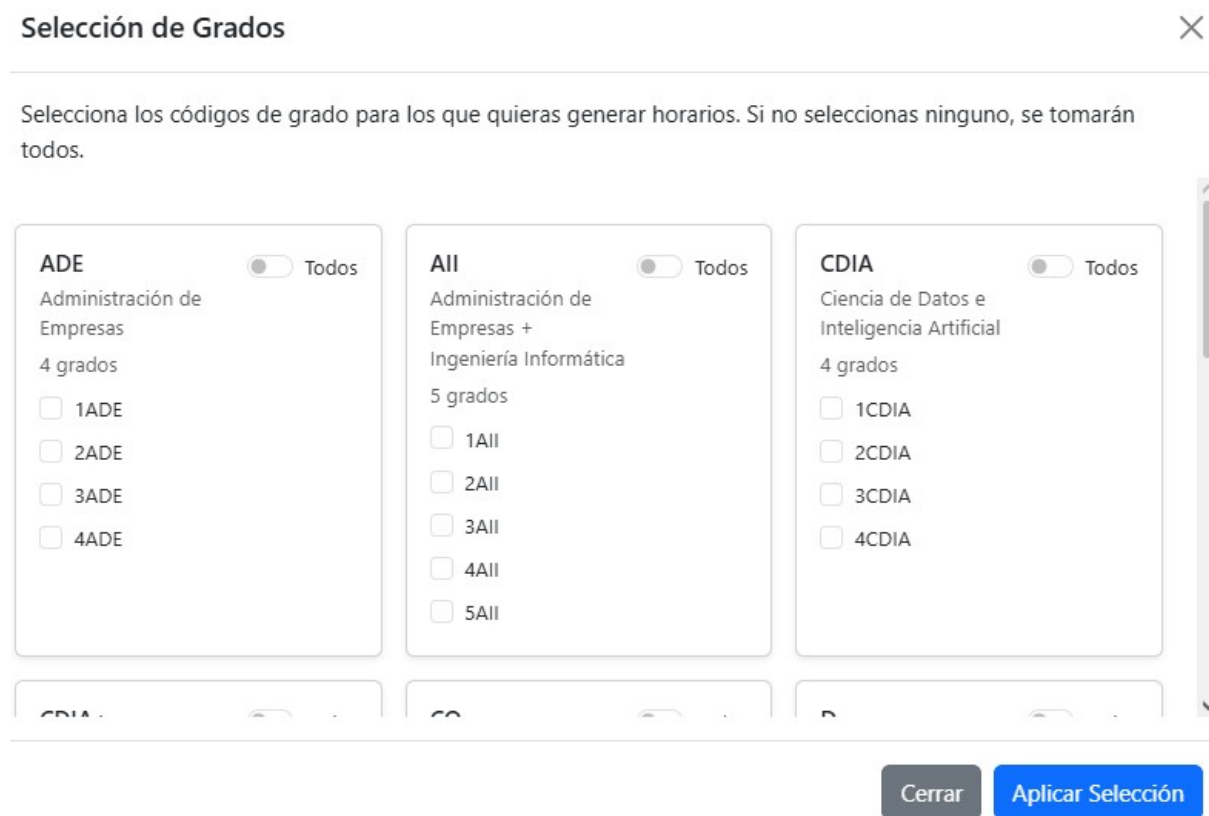
Minimizar Huecos

Optional

Iniciar Generación

Figura 5.4: Ventana de generación de horarios

Además, existe la posibilidad de seleccionar los grados y cursos de los que se quiera generar el horario, en caso de no querer generar un horario completo. Mediante el modal de selección de grados representado en la figura 5.5.



Selección de Grados ✕

Selecciona los códigos de grado para los que quieras generar horarios. Si no seleccionas ninguno, se tomarán todos.

ADE ● Todos

Administración de Empresas

4 grados

☐ 1ADE

☐ 2ADE

☐ 3ADE

☐ 4ADE

All ● Todos

Administración de Empresas + Ingeniería Informática

5 grados

☐ 1All

☐ 2All

☐ 3All

☐ 4All

☐ 5All

CDIA ● Todos

Ciencia de Datos e Inteligencia Artificial

4 grados

☐ 1CDIA

☐ 2CDIA

☐ 3CDIA

☐ 4CDIA

CDIA ● Todos

CDIA ● Todos

CDIA ● Todos

Cerrar Aplicar Selección

Figura 5.5: Modal de selección de grados

Tras iniciar el proceso, la aplicación ejecuta el algoritmo de generación y optimización descrito en capítulos anteriores, mostrando en todo momento en qué fase del proceso se encuentra. Dependiendo del tamaño del problema y de las restricciones seleccionadas, el tiempo de ejecución puede variar.

Una vez finalizada la ejecución, el sistema presenta las soluciones obtenidas mediante una representación visual organizada por días y franjas horarias. Mostrando los horarios agrupados por carreras y permitiendo navegar mediante botones entre los distintos cursos de las mismas, al igual que en la ventana de gestión de grados y asignaturas.

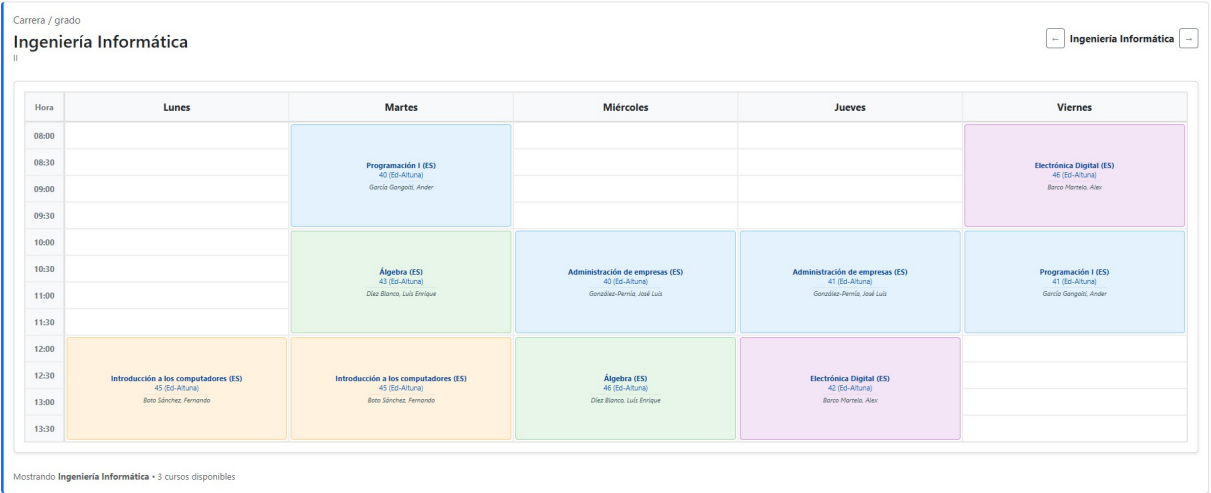


Figura 5.6: Visualización de horario de primer año de Ingeniería Informática

Además de la visualización gráfica del horario, el usuario puede consultar información adicional sobre las asignaciones realizadas, incluyendo las aulas utilizadas y el profesorado asignado a cada sesión. Esta representación facilita la validación de los resultados obtenidos y permite comparar distintas soluciones generadas por el sistema.

Por otra parte, una vez finalizado el proceso de generación y optimización, la aplicación muestra una auditoría de restricciones en la que se detalla la penalización acumulada por cada una de las restricciones blandas seleccionadas. Esta información proporciona contexto adicional sobre la calidad de la solución obtenida y ayuda al usuario a determinar si el horario generado resulta satisfactorio o si conviene ejecutar nuevamente el proceso para explorar alternativas diferentes.

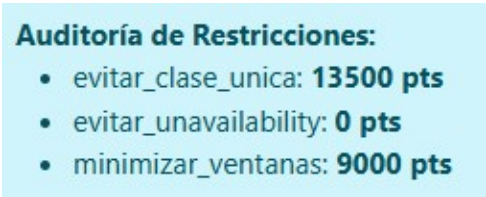


Figura 5.7: Auditoría de restricciones en una ejecución completa

6. TRABAJO FUTURO

El desarrollo realizado a lo largo de este proyecto ha permitido diseñar e implementar una aplicación capaz de generar horarios académicos de forma automática a partir de un conjunto de datos y restricciones configurables. Los resultados obtenidos demuestran la viabilidad del enfoque adoptado y permiten disponer de una herramienta funcional para apoyar las tareas de planificación académica.

No obstante, durante el desarrollo se identificaron diversas limitaciones y necesidades específicas que, debido a las restricciones de tiempo y alcance propias de un Trabajo Fin de Grado, no pudieron ser abordadas. Algunas de estas mejoras permitirían optimizar el rendimiento y la usabilidad de la aplicación, mientras que otras resultarían necesarias para adaptar completamente el sistema a los procedimientos y particularidades organizativas de la Universidad de Deusto.

En consecuencia, este capítulo recoge las principales líneas de trabajo futuro identificadas durante el desarrollo del proyecto, así como posibles ampliaciones que podrían incrementar la utilidad práctica de la herramienta en un entorno académico real.

6.1 SOPORTE PARA GRUPOS E IDIOMAS

Una de las principales simplificaciones realizadas durante el desarrollo del proyecto ha sido considerar que cada asignatura se imparte a un único grupo de estudiantes. Sin embargo, la organización académica de la Universidad de Deusto presenta situaciones más complejas que deberían ser contempladas en una futura evolución de la aplicación.

Por una parte, determinadas asignaturas cuentan con varios grupos paralelos debido al elevado número de estudiantes matriculados. Esta división permite repartir al alumnado entre diferentes grupos de una misma asignatura, reduciendo el tamaño de las clases y facilitando su gestión.

Por otra parte, algunas asignaturas pueden impartirse en diferentes idiomas, habitualmente español, euskera o inglés. En estos casos, cada estudiante selecciona el grupo correspondiente al idioma en el que desea cursar la asignatura, generando múltiples grupos asociados a una misma materia.

La incorporación de esta funcionalidad requeriría ampliar tanto el modelo de datos como las restricciones consideradas por el algoritmo de generación. En particular, deberían contemplarse dos situaciones diferenciadas.

La primera corresponde a aquellos casos en los que varios grupos de una misma asignatura son impartidos por un mismo docente. En este escenario, las distintas sesiones no podrían coincidir temporalmente, ya que el profesor únicamente puede impartir una clase en cada franja horaria. Por tanto, el sistema debería tratar estos grupos de forma independiente y garantizar que no se produzcan solapamientos.

La segunda situación aparece cuando los distintos grupos son impartidos por docentes diferentes. Dado que los estudiantes pertenecen a grupos distintos y únicamente asisten a una de las variantes disponibles, resultaría deseable que las sesiones se impartiesen simultáneamente siempre que fuese posible. De esta forma se mantendrían horarios más homogéneos entre estudiantes del mismo grado y curso, además de reducir las posibles interferencias con otras asignaturas.

La incorporación de estas características permitiría representar de forma más fiel la estructura académica real de la universidad y mejoraría significativamente la capacidad de adaptación de la aplicación a escenarios docentes complejos.

6.2 ASIGNACIÓN DE AULAS ESPECÍFICAS

La versión actual de la aplicación permite asociar a cada asignatura un conjunto de aulas compatibles que pueden utilizarse durante el proceso de generación. Este enfoque resulta suficiente para la mayoría de las sesiones teóricas, pero no permite representar determinadas necesidades docentes presentes en la planificación académica real de la Universidad de Deusto.

En algunas asignaturas, especialmente aquellas que incluyen actividades prácticas, es necesario impartir una parte de las sesiones en espacios especializados, como laboratorios informáticos, laboratorios técnicos o aulas equipadas con recursos específicos. En estos casos, no todas las sesiones semanales requieren el mismo tipo de aula, sino que únicamente determinadas sesiones deben realizarse obligatoriamente en un espacio concreto.

Para contemplar esta situación sería necesario ampliar el modelo de datos de las asignaturas, permitiendo especificar requisitos de aula diferenciados para cada una de las sesiones que las componen. De esta forma, una asignatura podría impartirse, por ejemplo, en un aula convencional durante una sesión semanal y requerir obligatoriamente un laboratorio durante la otra.

Desde el punto de vista del algoritmo de generación, esta funcionalidad implicaría incorporar nuevas restricciones que garantizaran la asignación de las sesiones prácticas a los espacios especializados correspondientes. Asimismo, sería necesario gestionar la disponibilidad limitada de este tipo de aulas, ya que suelen constituir

recursos compartidos entre múltiples titulaciones y asignaturas.

La incorporación de esta característica permitiría representar con mayor precisión la realidad académica de la universidad y facilitaría la generación de horarios más ajustados a las necesidades docentes de aquellas asignaturas que requieren equipamiento o infraestructuras específicas para su impartición.

6.3 GESTIÓN DE ASIGNATURAS OPTATIVAS

En la versión actual de la aplicación, las asignaturas marcadas como optativas son excluidas del proceso de generación automática de horarios. Esta decisión no responde a una limitación del algoritmo de generación en sí mismo, sino a las dificultades derivadas del modelo de datos empleado para representar la estructura académica.

A diferencia de las asignaturas obligatorias, cuya asistencia está asociada a grupos académicos claramente definidos, las asignaturas optativas presentan una casuística considerablemente más compleja. Un mismo estudiante puede elegir entre múltiples optativas disponibles dentro de su plan de estudios, lo que implica que dos asignaturas optativas podrían ser compatibles entre sí para algunos estudiantes e incompatibles para otros.

En determinadas titulaciones, además, el número de asignaturas optativas ofertadas supera ampliamente la capacidad temporal disponible dentro de un horario semanal. Como consecuencia, resulta inevitable que algunas de ellas deban impartirse simultáneamente. La dificultad reside en determinar qué asignaturas pueden solaparse sin generar conflictos para los estudiantes matriculados.

El modelo de datos desarrollado durante este proyecto únicamente contempla información relativa a asignaturas, docentes, aulas y grupos académicos. En consecuencia, el sistema no dispone de información suficiente para conocer qué estudiantes cursan cada asignatura optativa ni para identificar los posibles conflictos derivados de los solapamientos horarios.

La incorporación de esta funcionalidad requeriría una ampliación significativa del modelo de datos. Una posible solución consistiría en almacenar información individualizada sobre las matrículas de cada estudiante, permitiendo al sistema detectar automáticamente los conflictos entre asignaturas optativas. Alternativamente, podría diseñarse un sistema de agrupaciones académicas más avanzado que representase las relaciones de compatibilidad e incompatibilidad existentes entre las distintas optativas ofertadas.

Hasta que dicha información pueda ser incorporada al sistema, la generación automática de horarios para asignaturas optativas seguirá presentando limitaciones importantes. Por este motivo, en el contexto actual de la

aplicación, estas asignaturas deben continuar siendo planificadas manualmente por los responsables académicos, complementando posteriormente los horarios generados automáticamente para las asignaturas obligatorias.

6.4 GESTIÓN DE HORARIOS DE MAÑANA Y TARDE

Durante el desarrollo del proyecto se asumió que todas las actividades académicas podían programarse dentro de una única franja horaria común. Esta simplificación permitió reducir la complejidad inicial del problema y centrar el esfuerzo en la implementación del algoritmo de generación y en la gestión de las restricciones consideradas prioritarias.

Sin embargo, la organización docente de la Universidad de Deusto presenta una realidad más compleja. Existen titulaciones y cursos cuyas actividades se desarrollan principalmente en horario de mañana, mientras que otros concentran su docencia en horario de tarde. Asimismo, determinados cursos incluyen un elevado número de asignaturas prácticas u optativas, cuya planificación difiere de la estructura habitual seguida por las asignaturas obligatorias.

Debido al desconocimiento sobre qué titulaciones y cursos utilizan cada modalidad horaria, así como a la dificultad de representar adecuadamente las asignaturas optativas dentro del modelo de datos actual, esta característica quedó fuera del alcance del proyecto. Como consecuencia, el sistema no distingue actualmente entre asignaturas impartidas en horario de mañana o de tarde.

Una posible evolución de la aplicación consistiría en incorporar un nuevo atributo asociado a las asignaturas o grupos académicos que indicase la franja horaria en la que deben impartirse. Esta información podría representarse mediante un valor booleano o, de forma más flexible, mediante una etiqueta con valores como *morning* y *afternoon*.

Bajo esta aproximación, las asignaturas de mañana podrían programarse dentro del intervalo comprendido entre las 08:00 y las 16:30, manteniendo la pausa destinada a la comida entre las 14:00 y las 14:30. Por su parte, las asignaturas de tarde se desarrollarían aproximadamente entre las 14:30 y las 20:30.

La coexistencia de ambos modelos horarios genera una situación especialmente interesante en el intervalo comprendido entre las 14:30 y las 16:30, ya que se trata de una franja válida tanto para actividades de mañana como de tarde. Esta circunstancia permitiría utilizar dicho intervalo para programar asignaturas compartidas por estudiantes pertenecientes a ambas modalidades horarias.

Por el contrario, cuando una asignatura únicamente pertenezca a una de las dos modalidades, el algoritmo podría incorporar criterios de optimización específicos. En el caso de los estudios de mañana, resultaría deseable minimizar el uso de la franja compartida para favorecer que los estudiantes finalicen su jornada lectiva lo antes posible. De forma análoga, para los estudios de tarde podría priorizarse la utilización de dicho intervalo, permitiendo concentrar la actividad docente y reducir la duración total de la jornada académica.

La incorporación de esta funcionalidad permitiría adaptar con mayor precisión los horarios generados a la realidad organizativa de la universidad y mejoraría la calidad de las soluciones obtenidas en aquellos estudios que utilizan modelos docentes diferenciados entre mañana y tarde.

6.5 EXPORTACIÓN E INTEGRACIÓN DE HORARIOS

La versión actual de la aplicación está orientada principalmente a la generación y evaluación de soluciones. Una vez finalizado el proceso, los horarios pueden visualizarse a través de la interfaz web, pero no existe un mecanismo que permita almacenarlos de forma permanente o exportarlos para su utilización posterior.

Esta limitación implica que los horarios generados únicamente permanecen disponibles mientras la sesión de la aplicación continúa activa. Una vez cerrada la página o reiniciado el sistema, los resultados obtenidos dejan de estar accesibles, siendo necesario ejecutar nuevamente el proceso de generación, sin garantía de obtener la misma solución.

Como posible línea de mejora, resultaría interesante incorporar funcionalidades de exportación que permitieran descargar los horarios generados en formatos ampliamente utilizados, como PDF o hojas de cálculo. Esto facilitaría su distribución entre los responsables académicos y permitiría conservar diferentes alternativas de horario para su análisis y comparación posterior.

No obstante, una solución todavía más interesante consistiría en integrar la aplicación con los sistemas de gestión académica utilizados por la Universidad de Deusto. Mediante dicha integración, los horarios generados podrían exportarse directamente en un formato compatible con las herramientas empleadas actualmente por los responsables de planificación académica.

Este enfoque permitiría que la aplicación actuase como una herramienta de apoyo dentro del flujo de trabajo habitual de la universidad, facilitando la evaluación de horarios candidatos y reduciendo significativamente el trabajo manual necesario para trasladar la información entre sistemas. De esta forma, la utilidad práctica de la aplicación aumentaría considerablemente, acercándola a un escenario de utilización real dentro del entorno

universitario.

6.6 MEJORA DEL PROCESO DE IMPORTACIÓN DE DATOS

La aplicación desarrollada incorpora funcionalidades de importación de datos para aulas, docentes y asignaturas, permitiendo reducir significativamente el trabajo necesario para preparar la información empleada durante la generación de horarios. Estas funcionalidades fueron diseñadas tomando como referencia documentos reales proporcionados por los responsables de gestión académica de la Universidad de Deusto.

No obstante, los ficheros actualmente utilizados presentan una limitación importante: fueron concebidos como herramientas de consulta y gestión para uso humano, y no como formatos de intercambio de información entre sistemas. Como consecuencia, los datos contenidos en ellos requieren procesos adicionales de interpretación, adaptación y limpieza antes de poder ser utilizados correctamente por la aplicación.

Tal y como se ha comentado en el apartado 5.1, esta situación provoca que la importación de determinadas entidades, especialmente grados y asignaturas, no pueda realizarse de forma completamente automática y requiera una revisión posterior por parte del usuario. Aunque esta aproximación resulta suficiente para el alcance del proyecto, constituye una limitación relevante de cara a una posible implantación real de la herramienta.

Una posible línea de trabajo futuro consistiría en definir un modelo normalizado de hoja de cálculo específicamente diseñado para la importación de datos académicos. Dicho formato podría incorporar una estructura homogénea, validaciones previas y una organización de la información adaptada a las necesidades de la aplicación, permitiendo automatizar completamente el proceso de carga de datos.

Alternativamente, y en línea con la integración mencionada en la sección anterior, podría estudiarse la conexión directa con las fuentes de información institucionales utilizadas por la universidad. Esta aproximación eliminaría la necesidad de intercambiar archivos manualmente y permitiría mantener los datos utilizados por la aplicación sincronizados con la información académica oficial.

La incorporación de cualquiera de estas soluciones contribuiría a reducir la intervención manual requerida durante la preparación de los datos y mejoraría considerablemente la viabilidad de la aplicación como herramienta de uso cotidiano dentro de los procesos de planificación académica.

7. USO DE LA INTELIGENCIA ARTIFICIAL EN EL PROYECTO

Durante el desarrollo de este Trabajo Fin de Grado se utilizaron diversas herramientas basadas en inteligencia artificial como apoyo en distintas fases del proyecto. Estas herramientas se emplearon principalmente para facilitar tareas de investigación, resolver dudas técnicas, generar propuestas de implementación, revisar y generar fragmentos de código y mejorar la redacción de la documentación elaborada.

La utilización de estas herramientas tuvo como objetivo incrementar la productividad durante el desarrollo y servir como apoyo en la toma de decisiones técnicas. No obstante, todas las decisiones relacionadas con el diseño de la aplicación, la implementación de las funcionalidades, la validación de los resultados obtenidos y la elaboración final de la memoria fueron responsabilidad exclusiva del autor. Las respuestas y sugerencias proporcionadas por las herramientas de inteligencia artificial fueron analizadas críticamente y adaptadas cuando fue necesario para garantizar su adecuación a los objetivos y requisitos específicos del proyecto.

A continuación, se describen las principales herramientas de inteligencia artificial utilizadas, indicando el contexto en el que fueron empleadas y la finalidad de su uso dentro del proyecto.

7.1 CHATGPT

ChatGPT fue utilizado principalmente durante las fases de investigación y documentación del proyecto. Durante la etapa inicial de análisis, esta herramienta se empleó como apoyo para localizar recursos relevantes, identificar documentación técnica y obtener información introductoria sobre diferentes tecnologías, metodologías y enfoques relacionados con la generación automática de horarios académicos. Su utilización permitió agilizar la búsqueda de información y facilitar la exploración de posibles soluciones antes de consultar las fuentes oficiales correspondientes.

Adicionalmente, ChatGPT fue utilizado como herramienta de apoyo en la redacción de la memoria. En particular, se empleó para mejorar la claridad y la calidad de la expresión escrita de determinadas ideas previamente elaboradas por el autor, proponiendo alternativas de redacción, revisando la estructura de algunos apartados y ayudando a adaptar el texto a un estilo más formal y académico.

7.2 NOTEBOOKLM

NotebookLM fue utilizado como herramienta de apoyo durante la fase de investigación del proyecto. Para ello, se incorporaron diferentes documentos de referencia, incluyendo artículos científicos, trabajos relacionados y documentación técnica vinculada al problema de la generación automática de horarios académicos.

La herramienta se empleó principalmente para facilitar la consulta rápida de información contenida en estos documentos, permitiendo localizar conceptos, metodologías y resultados relevantes sin necesidad de revisar manualmente cada fuente en su totalidad. Asimismo, resultó especialmente útil para profundizar en aspectos específicos del problema estudiado, favoreciendo la comprensión de determinadas técnicas y enfoques presentes en la literatura especializada.

El uso de NotebookLM permitió agilizar el proceso de aprendizaje y análisis del estado del arte, facilitando la identificación de información relevante para el desarrollo del proyecto.

7.3 GEMINI

Gemini fue utilizado principalmente durante la fase de desarrollo de la aplicación como herramienta de apoyo para la implementación de funcionalidades y la generación de fragmentos de código. Su uso permitió obtener propuestas de implementación para determinadas tareas, así como ejemplos de código que posteriormente fueron revisados, adaptados e integrados en el proyecto según las necesidades específicas de la aplicación.

Además de su utilización como asistente de programación, Gemini desempeñó un papel relevante como herramienta de apoyo en la toma de decisiones relacionadas con el diseño del software. Durante el desarrollo, se empleó para analizar diferentes alternativas de implementación, evaluar posibles enfoques para resolver determinados problemas y debatir las ventajas e inconvenientes de distintas soluciones arquitectónicas y algorítmicas.

7.4 GITHUB COPILOT

GitHub Copilot fue utilizado durante la fase de desarrollo como asistente de programación integrado en el entorno de desarrollo Visual Studio Code. Gracias a su acceso al contexto completo del proyecto, esta herramienta proporcionó sugerencias de código adaptadas a la estructura, los modelos de datos y las funcionalidades ya implementadas en la aplicación.

Su uso resultó especialmente útil para agilizar la creación de fragmentos de código repetitivos o de carácter rutinario, tales como operaciones de creación, modificación y eliminación de datos, procesos de importación de información y otras funcionalidades con patrones de implementación similares. Asimismo, se empleó como apoyo para la detección y corrección de errores de programación, proponiendo modificaciones y alternativas para resolver incidencias identificadas durante el desarrollo.

Otra de las ventajas de la herramienta fue su capacidad para realizar modificaciones rápidas sobre diferentes elementos del proyecto aprovechando el contexto disponible dentro del editor. Esto permitió automatizar parcialmente determinadas tareas de refactorización y adaptación de código, reduciendo el tiempo necesario para realizar cambios repetitivos en múltiples archivos.

8. VALORACIÓN ÉTICA DEL PROYECTO

El desarrollo de herramientas informáticas destinadas a automatizar procesos de planificación académica puede generar importantes beneficios para las instituciones educativas, pero también plantea determinadas cuestiones éticas que deben ser consideradas. Aunque la aplicación desarrollada en este proyecto tiene como objetivo principal asistir a los responsables académicos en la generación de horarios, las decisiones tomadas durante el diseño del sistema pueden influir indirectamente en la distribución de recursos, en la calidad de los horarios obtenidos y en la experiencia de los distintos colectivos afectados.

Por este motivo, resulta necesario analizar tanto los beneficios potenciales que puede aportar la aplicación como los posibles riesgos o limitaciones derivados de su utilización.

8.1 BENEFICIOS POTENCIALES

La implantación de una herramienta de generación automática de horarios puede aportar diversas ventajas desde el punto de vista organizativo y social:

- **Reducción de la carga de trabajo administrativa.** La elaboración manual de horarios académicos es una tarea compleja y laboriosa que requiere considerar una gran cantidad de restricciones y dependencias. La automatización de este proceso puede reducir significativamente el tiempo y el esfuerzo necesarios para realizar esta planificación.
- **Disminución del estrés asociado a la planificación académica.** El proceso de generación manual de horarios universitarios puede causar un gran estrés en los responsables, debido a la complejidad que dicho problema supone. Al delegar parte del proceso de generación en una herramienta informática, los responsables académicos pueden centrarse en la evaluación y validación de las soluciones propuestas en lugar de dedicar gran parte de su tiempo a construir los horarios manualmente.
- **Mejora de la calidad de los horarios.** La consideración sistemática de restricciones y preferencias permite generar soluciones que reduzcan situaciones poco deseables para los estudiantes, como jornadas excesivamente largas, huecos innecesarios entre clases o distribuciones poco equilibradas de la carga lectiva.
- **Evaluación objetiva de alternativas.** La generación automática facilita la obtención y comparación de múltiples soluciones candidatas, permitiendo seleccionar aquella que mejor se adapte a los criterios es-

tablecidos por la institución.

8.2 RIESGOS Y CONSIDERACIONES ÉTICAS

A pesar de los beneficios descritos, la utilización de este tipo de sistemas también puede plantear determinados riesgos que conviene tener en cuenta.

- **Dependencia excesiva de la herramienta.** Existe el riesgo de asumir que los horarios generados son necesariamente correctos o adecuados por el simple hecho de haber sido producidos por un sistema automatizado. Por ello, la aplicación debe considerarse una herramienta de apoyo a la toma de decisiones y no un sustituto de la supervisión humana.
- **Falta de equidad en la distribución de penalizaciones.** El algoritmo implementado busca minimizar la penalización total asociada a las restricciones blandas. Sin embargo, este enfoque no garantiza que dichas penalizaciones se distribuyan de forma equilibrada entre todos los grados, cursos o colectivos afectados. Como consecuencia, podrían generarse horarios considerados válidos desde el punto de vista global, pero percibidos como injustos por concentrar un número elevado de inconvenientes en un grupo concreto de estudiantes.
- **Posible utilización de datos sensibles en futuras ampliaciones.** La versión actual de la aplicación trabaja principalmente con información académica relativa a asignaturas, docentes y aulas. No obstante, futuras extensiones podrían requerir información más detallada sobre estudiantes individuales o sus preferencias. En ese escenario, sería necesario garantizar el cumplimiento de la normativa de protección de datos y aplicar medidas adecuadas para preservar la privacidad de las personas afectadas.

8.3 REFLEXIÓN FINAL

La aplicación desarrollada debe entenderse como una herramienta de apoyo destinada a facilitar el trabajo de planificación académica y no como un sistema capaz de sustituir completamente el criterio humano. La supervisión de los resultados obtenidos continúa siendo necesaria para garantizar que los horarios generados no solo sean válidos desde un punto de vista técnico, sino también adecuados, equilibrados y justos para los distintos colectivos implicados.

Por tanto, el uso responsable de este tipo de herramientas requiere combinar las ventajas de la automatización con la experiencia y capacidad de evaluación de los responsables académicos encargados de la planificación.

9. CONCLUSIONES

La realización de este Trabajo Fin de Grado ha permitido abordar un problema cuya complejidad resulta considerablemente mayor de lo que puede parecer a primera vista. La planificación y generación de horarios académicos es una tarea con la que estudiantes y docentes conviven diariamente, lo que puede llevar a percibirla como un proceso relativamente sencillo. Sin embargo, el desarrollo de este proyecto ha puesto de manifiesto la gran cantidad de restricciones, dependencias y criterios que deben ser considerados para obtener soluciones viables y de calidad.

Desde un punto de vista técnico, el proyecto ha supuesto una oportunidad para profundizar en áreas como el diseño de algoritmos, la optimización de problemas combinatorios, el desarrollo de aplicaciones web y la gestión de datos. Asimismo, ha permitido comprender las dificultades asociadas a la modelización de problemas reales, donde no siempre existe una única solución correcta y donde es necesario equilibrar múltiples objetivos y restricciones, en ocasiones contrapuestos entre sí.

El resultado obtenido puede considerarse satisfactorio, ya que se ha logrado desarrollar una aplicación funcional capaz de generar automáticamente horarios académicos válidos a partir de la información y restricciones definidas por el usuario. La aplicación incorpora además herramientas para la gestión de los datos necesarios, la visualización de los horarios generados y el análisis de la calidad de las soluciones obtenidas, cumpliendo así los objetivos principales planteados al inicio del proyecto.

No obstante, durante el desarrollo también se han identificado diversas limitaciones. La aplicación actual no contempla todos los escenarios y particularidades necesarias para integrarse directamente en los procesos de planificación académica de la Universidad de Deusto. Existen funcionalidades, restricciones y casos de uso específicos que requerirían un desarrollo adicional para alcanzar un nivel de madurez suficiente para su utilización en un entorno real de producción.

A pesar de estas limitaciones, se considera que el proyecto ha cumplido con éxito su propósito fundamental. Además de proporcionar una base funcional sobre la que continuar desarrollando nuevas capacidades, el trabajo realizado ha permitido analizar en profundidad el problema específico de generación de horarios dentro del contexto de la Universidad de Deusto, identificando tanto sus desafíos como posibles líneas de evolución futura.

En definitiva, este proyecto ha permitido demostrar la viabilidad de aplicar técnicas de generación automática de horarios al ámbito universitario, al mismo tiempo que ha proporcionado una valiosa experiencia formativa

tanto en el plano técnico como en el metodológico. El conocimiento adquirido y la solución desarrollada constituyen una base sólida para futuras mejoras y para una posible evolución hacia una herramienta con aplicación práctica dentro del entorno universitario.

10. REPOSITORIO DEL PROYECTO

El código fuente desarrollado en este Trabajo de Fin de Grado se encuentra disponible en un repositorio público de GitHub, con el objetivo de facilitar la consulta, reproducción y posible extensión del sistema implementado.

<https://github.com/Gontzaa86/PFG-25-26>

BIBLIOGRAFÍA

- [1] “Automated timetabling: The state of the art,” <https://es.scribd.com/document/354985608/State-of-Art>, documento consultado en Scribd, accedido 16 de febrero de 2026.
- [2] “Investigación de operaciones,” https://es.wikipedia.org/wiki/Investigaci3n_de_operaciones, wikipedia, accedido 10 de enero de 2026.
- [3] “A survey of approaches for university course timetabling problem,” https://www.researchgate.net/publication/351120554_A_SURVEY_OF_APPROACHES_FOR_UNIVERSITY_COURSE_TIMETABLING_PROBLEM, accedido 16 de febrero de 2026.
- [4] “Coloración de grafos,” https://es.wikipedia.org/wiki/Coloraci3n_de_grafos, wikipedia, accedido 18 de febrero de 2026.
- [5] “Programación lineal,” https://es.wikipedia.org/wiki/Programaci3n_lineal, wikipedia, accedido 18 de febrero de 2026.
- [6] “Integer programming,” https://en.wikipedia.org/wiki/Integer_programming, wikipedia, accedido 18 de febrero de 2026.
- [7] “Problema de satisfacción de restricciones,” https://es.wikipedia.org/wiki/Problema_de_satisfacci3n_de_restricciones, wikipedia, accedido 07 de enero de 2026.
- [8] “Metaheurística,” <https://es.wikipedia.org/wiki/Metaheur3stica>, wikipedia, accedido 19 de febrero de 2026.
- [9] “Hyper-heuristics: a survey of the state of art,” <https://link.springer.com/article/10.1057/jors.2013.71>, accedido 16 de febrero de 2026.
- [10] “Itc 2019: International timetabling competition,” <https://www.itc2019.org/home>, (accedido 28 de febrero de 2026).
- [11] UniTime Project, “Unitime - university timetabling system,” <https://www.unitime.org>, (accedido 11 de febrero de 2026).
- [12] “Fet (timetabling software),” https://en.wikipedia.org/wiki/FET_%28timetabling_software%29, (Wikiped-

dia, accedido 07 de enero de 2026).

- [13] Liviu Lalescu, “Fet: Free timetabling software,” <https://lalescu.ro/liviu/fet/>, (accedido accedido 07 de enero de 2026).
- [14] Apache Software Foundation, “Optaplanner - constraint satisfaction solver,” <https://kie.apache.org/components/optaplanner>, (accedido 11 de febrero de 2026).
- [15] Untis GmbH, “Untis - software de horarios escolares y universitarios,” <https://www.untis.at/es>, (accedido 7 de febrero de 2026).
- [16] ASC Timetables, “Asc timetables - software de planificación de horarios,” <https://www.asctimetables.com>, (accedido 7 de febrero de 2026).
- [17] Horarium AI, “Horarium - generación automática de horarios con ia,” <https://horarium.ai/es>, (accedido 7 de febrero de 2026).
- [18] Omniscol, “Omniscol - software de gestión de horarios escolares,” <https://www.omniscol.com>, (accedido 7 de febrero de 2026).
- [19] “Scrum (desarrollo de software),” https://es.wikipedia.org/wiki/Scrum_%28desarrollo_de_software%29, wikipedia, accedido 11 de junio de 2026.
- [20] Pallets Projects, “Flask documentation,” <https://flask.palletsprojects.com/en/stable/>, (accedido 20 de marzo de 2026).
- [21] Bootstrap Team, “Bootstrap 5.3 documentation - introduction,” <https://getbootstrap.com/docs/5.3/getting-started/introduction/>, (accedido 30 de marzo de 2026).
- [22] UniTime Project, “Unitime university course timetabling data format v24,” https://www.unitime.org/uct_dataformat_v24.php, (accedido 15 de marzo de 2026).
- [23] “Backtracking,” https://es.wikipedia.org/wiki/Vuelta_atrÃas, (Wikipedia, accedido 22 de febrero de 2026).